



SCHOOL OF
**COMPUTING &
DATA SCIENCE**
The University of Hong Kong



HKU SAAS
Data Science Lab

Financial Chatbot: Building An Ontology-Based AI Investment Chatbot

APAI 3799 Final Year Project

Students: Chris Tang Patel Harshvardha Hui Lok Samuel

Supervisor: Dr. LAU, Adela Sau Mui

School: School of Computing and Data Science
The University of Hong Kong

Date: May 2026

School of Computing and Data Science, The University of Hong Kong

Abstract

Financial markets are dynamic ecosystems governed by hierarchical relationships between macroeconomic sectors and individual corporate entities. Although Large Language Models (LLMs) demonstrate exceptional natural language generation, they are unreliable financial advisors for three structural reasons: (i) they process market data as flat unstructured text, whereby critical hierarchical sector-to-stock relationships are lost; (ii) they rely on static training data, leading to temporal staleness and hallucinations regarding current market metrics and live prices; and (iii) they suffer from opaque reasoning, offering no audit trails to inspect which sources were consulted or how answers were assembled for high-stakes decisions. We address these limitations with the **Financial Ontology Chatbot**, a production-grade conversational agent that fuses three previously separated lines of work: (1) a proprietary five-domain Financial Knowledge Graph that encodes match/invert directional logic across the 17-leaf relationship hierarchy; (2) an agentic backend that routes user queries to a fleet of stealth real-time scrapers, an asynchronous labelling pipeline, and a stacking ensemble (LSTM + GBM + meta-learner) for directional prediction; and (3) a decoupled Next.js frontend that streams the model’s reasoning steps over Server-Sent Events and renders mode-specific dashboards. Trained on 1.36M LLM-labelled headlines across 146 tickers and 24 sectors, the ontology-adjusted ensemble achieves **64.41 % accuracy**, **F1 = 0.6709**, and **AUC = 0.7514** on the 1-day directional task at threshold 0.42—an absolute gain of **+3.04 % accuracy**, **+2.19 % F1**, and **+4.85 % AUC** over an otherwise-identical raw-sentiment baseline. This demonstrates that injecting structured economic relationships into the sentiment signal materially improves directional prediction, and that streaming agentic reasoning makes domain-specialised LLM advice transparent and auditable. Code is available at: <https://github.com/IILKA/RetailLM/>.

Contents

1	Introduction	4
2	Literature Review	5
2.1	Financial Sentiment Analysis	5
2.2	LSTM and Deep Learning for Stock Prediction	5
2.3	Large Language Model Agents and Tool Use	5
2.4	Financial Knowledge Graphs and Ontologies	5
2.5	AI Agents in Financial Services and Research Gaps	6
3	Methodology	6
3.1	System Architecture	6
3.2	Pre-training Data Acquisition	8
3.2.1	Design Objectives & Criteria	8
3.2.2	System Reliability & Storage Architecture	8
3.2.3	News Scraping Parameters and Strategy	9
3.2.4	Structured Feature Generation: FinBERT and Gemini 2.5 Flash Lite . . .	10
3.2.5	Price Extraction and Temporal Alignment	10
3.3	Real-Time Scraping Agents	10
3.4	Financial Knowledge Graph and Ontology	12
3.4.1	Five-Domain Hierarchy	12
3.4.2	Directional Multiplier: Match vs. Invert	13
3.4.3	Design Rationale: Determinism and Hierarchy	14
3.5	LLM Labeller: Gemini 2.5 Flash Lite with Structured Outputs	14
3.6	Sentiment Analysis Pipeline	15
3.6.1	FinBERT Scoring with Lexicon Fallback	15
3.6.2	Ontology Adjustment	15
3.6.3	Daily Aggregation and Lag Features	15

3.7	Price Feature Engineering	16
3.8	Target Variable and Prediction Horizons	17
3.9	Predictive Model: LSTM + GBM + Stacking Ensemble	17
3.9.1	Modeling Iteration History	17
3.9.2	Two-Branch LSTM	17
3.9.3	Gradient Boosting Machine	19
3.9.4	Stacking Meta-Learner	19
3.9.5	Why an Ensemble?	20
3.9.6	Threshold Tuning	20
3.9.7	Loaded Artefacts	21
3.9.8	Model Size, Inference Cost, and Latency Profile	21
3.10	Web UI	21
4	Experiments and Results	22
4.1	Dataset Characterization	22
4.2	Evaluation Metrics	23
4.3	Experimental Protocol	23
4.4	Pre-training Scraper Benchmarks	24
4.5	Real-time Agent Performance	24
4.6	Predictive Model Evaluation: Multi-Horizon Results	24
4.6.1	Discussion of Multi-Horizon Behaviour	25
4.7	Threshold Tuning	25
4.8	Ablation: Contribution of Sentiment Features	26
4.9	Ontology Adjustment Impact	26
4.10	Chatbot End-to-End Capability	27
5	Discussion	28
5.1	Why Does the 3-day Horizon Outperform 1-day?	28
5.2	Why Does the LSTM Beat the GBM on F1?	28
5.3	Why Does the Ontology Help Even Though Sentiment Alone Doesn't?	28
5.4	Limitations	28
5.5	Comparison with Prior Work	29
6	Conclusions and Future Work	29
A	Appendix Overview	31
B	Data Samples	31
B.1	Unlabelled Scraped News Data	31
B.2	Labelled Data Samples with Reference	31
C	API Reference	32
C.1	POST /api/chat/stream (Server-Sent Events)	32
C.2	POST /api/chat (Legacy JSON)	33
C.3	GET /api/health	33
C.4	POST /api/log	33
D	Scraper Mode Reference	33

E	Full LLM Prompts	35
E.1	Intent Routing Prompt	35
E.2	Response-Generation System Prompt	35
E.3	News Labelling System Prompt	35
E.4	Mode-Specific Response Templates	36
F	Stock-Analysis Pipeline Data Flow	36
G	System Architecture Diagrams	37
G.1	Request Lifecycle: Sequence Diagram	37
G.2	ML Pipeline: Stock-Analysis Flowchart	38
G.3	SSE Streaming Flow	39
H	Project Repository	40
I	Web Interface Examples	40
J	Division of Labour	47

1 Introduction

Algorithmic trading and quantitative analysis have long depended on the synthesis of large volumes of heterogeneous data: historical prices, intraday volumes, fundamental ratios, and—increasingly—real-time news sentiment [2, 3]. Stock prices rarely move in isolation. A factory fire at a competitor cascades into market-share gains for the target firm; a chip-process breakthrough at a supplier compresses the input costs of every downstream OEM; a regulatory ban on a substitute product shifts demand to incumbents. Capturing these directional cascades requires more than processing news as a string of tokens—it requires a *hierarchical* model of the market.

The recent generation of Large Language Models (LLMs) excels at fluent natural-language generation but is structurally ill-suited to this task. Three failure modes recur in our evaluations: *temporal staleness* (pre-trained weights freeze at a cutoff date, so “What is BYD trading at today?” produces fabricated or refused responses), *flat semantic processing* (a vanilla LLM scoring “Competitor’s Shanghai factory destroyed by fire” returns strongly negative sentiment because the tokens are negative, even though the event is bullish for the target firm capturing lost market share [4, 5]), and *opaque reasoning* (end-users cannot inspect which data sources were consulted or how they were combined). Recent surveys of consumer behaviour underscore the cost: [20] reports that 55 % of UK adults consult AI platforms for financial guidance at least sometimes (81 % among Gen-Z), while 19 % of US adults—and 27 % of Gen-Z investors—report losing more than \$100 by following AI-generated financial advice. The asymmetry between adoption and reliability motivates a system architecture that grounds every claim in retrievable evidence.

This paper introduces the **Financial Ontology Chatbot**, an end-to-end conversational agent that addresses all three failure modes simultaneously. Our contributions are:

1. **Hierarchical Financial Knowledge Graph.** A five-domain ontology—Self, Horizontal, Vertical, Strategic, and Market-Microstructure—with deterministic match/invert directional multipliers that translate tokenised sentiment into directionally correct trading signals (Section 3.4).
2. **Agentic dual-data pipeline.** A high-throughput news scraper that achieves **0.0288 s/item** via headline-only extraction for training data (a 300-fold speed-up over full-page DOM parsing), paired with a fleet of stealth real-time news and price scraping agents using Playwright with anti-bot fingerprint masking and multi-platform fallback chains (Sections 3.2 and 3.3).
3. **Structured-output LLM labelling.** A Gemini 2.5 Flash Lite labeller bound by an OpenAPI `response_schema` that eliminates JSON parsing failures, with bounded-concurrency `asyncio` execution at five parallel calls per ticker (Section 3.5).
4. **Hybrid stacking ensemble.** An LSTM with bidirectional attention on a 20-day price feature + 3-day sentiment mean lookback, a Gradient Boosting Machine on flattened 72-feature aggregates, and a Gradient-Boosting meta-learner—all trained on 1.37 M samples across 146 Hong Kong tickers (Section 3.9).
5. **Web UI.** A Next.js 16 frontend that consumes the backend over Server-Sent Events, exposing every pipeline step live to the user alongside mode-specific interactive dashboards and tree-based chat history. (Section 3.10).
6. **Empirically validated ontology gains.** The ontology-adjusted ensemble outperforms the raw-sentiment baseline by **+3.04 %** accuracy, **+2.19 %** F1, and **+4.85 %** AUC at threshold 0.42, demonstrating that explicit relationship modelling adds signal beyond what sentiment scoring alone provides.

2 Literature Review

2.1 Financial Sentiment Analysis

The application of sentiment analysis to financial markets has matured rapidly with transformer architectures. Early work used lexicon-based methods (Loughran–McDonald [12]) and classical ML classifiers; BERT and its domain-adapted descendants marked a paradigm shift. Domain-fine-tuned models such as FinBERT [6] consistently outperform general-purpose architectures on short-form financial headlines, where surrounding linguistic context is sparse. FinBERT scores have been embedded into multi-factor asset-pricing models, most notably the Fama–French five-factor framework [15], revealing time-varying sensitivity where sentiment has positive impact during normal regimes and amplified or reversed effects under volatility shocks. Large-scale studies covering multi-year horizons confirm that incorporating daily news sentiment improves accuracy beyond a price-only baseline, particularly when fused non-linearly with price via joint sequential modelling rather than late fusion. Despite these advances, prior work treats sentiment as a *scalar* signal: a single signed score per article. Our work departs from this convention by introducing a *directional* multiplier (match/invert) anchored to economic relationships, operationally inverting the sentiment of competitor news and matching that of supply-chain or strategic news.

2.2 LSTM and Deep Learning for Stock Prediction

Long Short-Term Memory (LSTM) networks [7] are a cornerstone of financial time-series forecasting because they capture long-range temporal dependencies and mitigate the vanishing-gradient problem of classical RNNs. Recent comparative work has examined LSTM versus pure transformer architectures [8] for stock prediction; transformers excel on long sequences via parallel attention, but for the daily-frequency, 20-day-lookback regime that dominates practitioner workflows, LSTMs remain computationally efficient and empirically more stable. Bidirectional LSTMs with self-attention—the configuration adopted in our work—combine the temporal inductive bias of recurrence with selective focus and outperform unidirectional baselines on Hong Kong equity data. Multimodal fusion is an active frontier: two-branch architectures (separate sub-networks for price and sentiment, late-concatenated) have shown promise. Our system follows this design and extends it with a stacking ensemble combining the LSTM with a Gradient Boosting Machine on flattened aggregates, exploiting the complementarity between sequential and statistical views of the same data.

2.3 Large Language Model Agents and Tool Use

LLM-based goal-driven agents that plan, invoke tools, and adapt to feedback are increasingly viewed as a path to general-purpose assistants [9, 10]. Tool integration—the ability of an LLM to call calculators, code interpreters, search APIs, or domain-specific scrapers—is the operational mechanism, and standardised paradigms exist for mapping user instructions into executable plans [9]. Surveys of finance benchmarks identify recurring weaknesses including limited numerical reasoning, prompt sensitivity, and absent real-time adaptation. Our architecture addresses the last two: prompts are short, templated, and schema-enforced; every user query triggers fresh scraper invocations rather than relying on cached knowledge.

2.4 Financial Knowledge Graphs and Ontologies

Knowledge graphs (KGs) have gained traction in finance for fraud detection, anti-money-laundering, credit-risk assessment, and regulatory compliance [11]. An explicit ontology atop a KG addresses a structural limitation of mono-relational graphs: their inability to express *kind-of* and *instance-of* hierarchies needed for commonsense reasoning. Comparative studies of financial causality

KGs report broader entity coverage and richer causal reasoning when ontologies are present [1]. The combination of KGs with LLMs is particularly active: recent FintechKG-style constructions extract commercial entities, conceptual entities, and temporal dimensions and propagate them through RGCN/LSTM hybrids for performance prediction [11]. Our contribution is orthogonal: rather than embedding the KG as input features, we use the ontology as a *deterministic post-processor* on FinBERT-scored sentiment, applying a directional multiplier (match/invert) that encodes the zero-sum nature of competitive markets. This preserves FinBERT’s interpretability while injecting domain logic where it has the largest empirical effect.

2.5 AI Agents in Financial Services and Research Gaps

Reviews of agent-based AI in finance [19] describe rapid progress across trading, fraud detection, robo-advisory, and compliance, with RL, multi-agent systems, and LLM-based agents progressively displacing simpler statistical models. Yet the same surveys flag a persistent gap between LLM fluency and grounding: without verifiable retrieval, agents fabricate [4, 5]. Three gaps in the literature directly motivate our architecture. *First*, sentiment analysis (FinBERT) and time-series prediction (LSTM) have been studied in isolation; their integration within an ontology-driven framework that explicitly models economic relationships—competitor, supplier, customer, partner, index member—remains under-explored, with prior work treating sentiment as a scalar and missing the directional nuance captured by our zero-sum/competitor inversion. *Second*, the integration of knowledge graphs with real-time tool-using LLM agents for conversational investment advice is nascent. *Third*, domain-specific instantiations combining structured ontologies, dual-data pipelines (historical + real-time), and hybrid predictive engines for finance are scarce. Our work fills these gaps with an end-to-end production architecture that binds every advisory utterance to a JSON payload retrieved during the same conversational turn.

3 Methodology

3.1 System Architecture

Illustrated in Figure 1, the Financial Ontology Chatbot executes its function through a four-phase pipeline. Appendix G (Figure 6) contains a complete representation of the entire request lifecycle through all actors involved (Client, Server, LLM). User Prompts progress through four phases: (1) Intent Routing; (2) Parallel Tool Execution with fallback; (3) Aggregation & Response Generation; and (4) Multi-Modal UI Rendering.

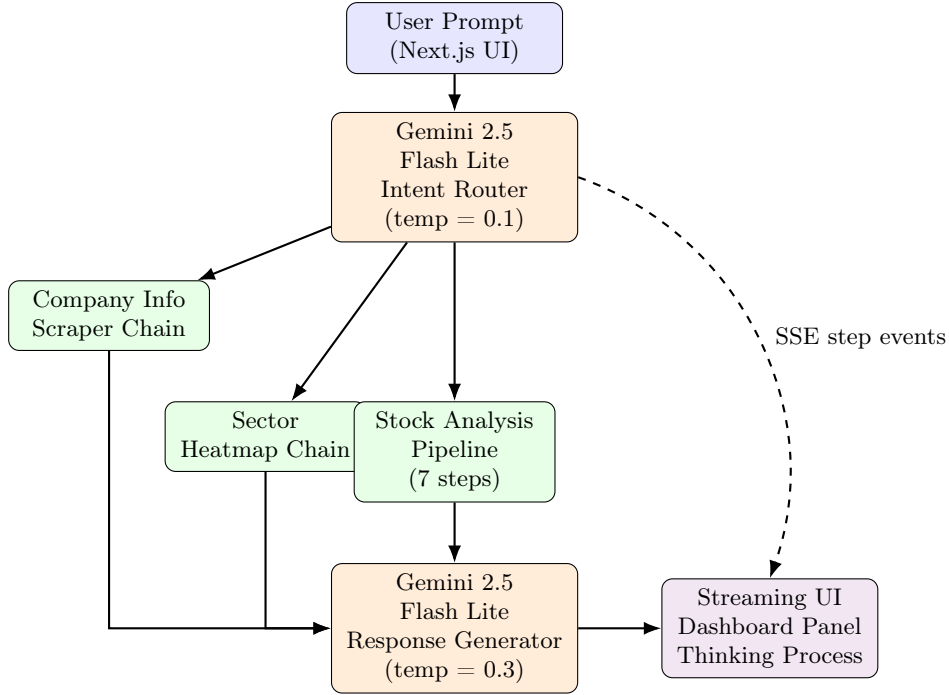


Figure 1: The Agentic execution pipeline consist of three tool chains; Each tool chain produces typed step events which are streamed to the user in real time via Server-Sent Events. The Response Generator (a second Gemini call) will take the user input and the JSON payload output by the tool chain to produce the final response, as well as generating a dashboard where data is generated using the output of agentic tools

Phase 1 — Intent Routing. Accomplished by a lightweight, low-temperature Gemini 2.5 Flash Lite API call to classify the prompt into one of four modes: `company_info`, `sector_analysis`, `stock_analysis`, or `none`. The routing prompt (see Appendix E) will return a strict JSON object with the chosen mode, the extracted stock or sector name, and a single sentence stating the reason why that mode was chosen. Temperature is held at 0.1 to ensure that the routing is deterministic.

Phase 2 — Tool Execution with Fallback. The relevant tool chain will be invoked to execute the required task, with each tool chain containing a primary scrapping component and a series of fallbacks. `company_info` runs Futunn → TradingView (reversed for US-listed tickers via the suffix of the mapped symbol); `sector_analysis` runs TradingView → Futunn → Yahoo Finance; `stock_analysis` runs a seven-step pipeline (news scraping → LLM labelling → sentiment scoring → ontology adjustment → daily aggregation → price features → ensemble prediction; see the detailed flowchart in Figure 7). All Playwright-based scrapers use the synchronous API and are offloaded to a thread pool by `asyncio.to_thread()` to prevent blocking the FastAPI event loop.

Phase 3 — Aggregation and Response Generation. The second Gemini 2.5 Flash Lite call (with temperature 0.3) will process the User message, as well as the output of the agentic tools in a compressed JSON Tool Payload to accomplish this. The system instruction (see Appendix E) prevents the model from using fake numbers to provide concise 2–4-sentence answers. The Mode-specific response templates ensure the model to tailors its output according to the user mode, delivering the most relevant and useful information.

Phase 4 — UI Rendering and Streaming. The Backend response is separated into two parts: a `reply_text` string and a typed `dashboard_payload` (`CompanyInfoPayload`, `SectorPayload`, `StockAnalysisPayload`, or `null`). The frontend selects the appropriate renderer based on the `type` field of the discriminated union. The streaming endpoint `POST /api/chat/stream` wraps the same pipeline as an asynchronous generator to yield typed `ThinkingStep` events at each phase boundary, framed as SSE blocks (See Figure 8 for an illustration of the Client to Server Streaming Flow). This will convert an opaque wait into a transparent and auditable progression. Taking stock analysis mode as an example, the pipeline is: `intent_routing` → `tool_selection` → `news_scraping` → `llm_labeling` → `sentiment` → `ontology` → `daily_agg` → `price_fetch` → `model` → `response_generation`.

3.2 Pre-training Data Acquisition

The efficacy of a domain-specific financial system is bounded by the quality of its training corpus. We engineered a comprehensive pre-training pipeline comprising three core components: a high-throughput historical *news* scraper, a high-velocity LLM *labelling* pipeline, and a structured *price* extractor for the OHLCV time-series.

3.2.1 Design Objectives & Criteria

The primary goal was to develop a dataset that combines quantitative financial information with qualitative market sentiment. To create this dataset, there needs to be a news scraping and price extraction pipeline that meets four criteria:

- **Historical Depth:** Allowing for longer term trends than would otherwise be possible with current real-time APIs.
- **Textual Signal Density:** Provide accurate representations of market sentiment, without including the additional, and often obscuring, semantic noise resulting from the full-length HTML bodies of news articles.
- **Numerical Data Integrity:** Structured market price data (Open, High, Low, Close, and Volume) from multiple trade/price feeds that can then be temporally aligned with the corresponding semantic news events.
- **Fault Tolerance & Throughput:** Resilience against network timeouts, evasion of sophisticated bot-detection heuristics, and high-speed data retrieval to support large-scale ontology training.

3.2.2 System Reliability & Storage Architecture

Given the volume of data required, the scraper was engineered for high-throughput, long-running execution. Key architectural and reliability features include:

- **Smart Resume (State Persistence):** A checkpointing system logs progress in case of an interruption. The scraper is capable of resuming from the last successfully processed stock rather than restarting.
- **Crash Resilience:** Error-handling for network timeouts and connection resets to prevent hindering the global execution loop (ability to complete multiple requests from the smokers without interruption)
- **Performance Optimization (Concurrency):** To maximize data throughput, the scraper uses asynchronous execution and multi-threading frameworks in the extraction pipeline to maximize throughput. The scraper can concurrently fetch and parse data from multiple stock tickers which significantly decreases the overall time taken to create the pre-training corpus.

- **Lightweight Storage Architecture:** To minimize I/O bottlenecks and simplify integration into downstream pipelines, the scraper does not utilize complex database management systems. The cleaned, extracted data points are serialized as well as directly stored in simple jsonl files (`.jsonl`), ensuring high-speed writes during high-frequency scraping loops.

Eventually, the scraper’s pipelines produced a temporally-aligned corpus of raw text from 146 tickers listed on the Hong Kong Stock Exchange. After data cleaning and feature extraction, there was a total of 1.37 million daily samples and 213,083 fully labeled and processed headlines available for model training.

3.2.3 News Scraping Parameters and Strategy

During the design phase, several target parameters were evaluated for the extraction of news information, including the full news article content, source URLs, author ratings, publication dates, and headlines. However, empirical evidence indicated that long-format articles generally have a distorting effect on the core signal associated with market sentiment, and cause a considerable increase in the latency associated with scraping. As a result, some parameters were eliminated.

The final strategy aims at collecting high-fidelity semantic signals to determine short-term market movements:

- **News Title:** Acts as the primary semantic feature, offering condensed summary of the market-related event without the input from the variations present in the HTML markup that comprises the article body.
- **Absolute Publication Date:** Essential for precise time-series alignment with historical stock prices. Sources that obfuscate the date (e.g., by indicating "3 months ago") will not be used in the extraction; therefore, sources that provide the publication date in the form YYYY-MM-DD directly on their index pages will be used.

The scraper architecture was developed iteratively in order to achieve the specified levels of accuracy and reliability as a result of this development. The first four iterations of the scraper were rejected for a number of reasons: the yfinance API was determined to be deficient in terms of historical depth when extracting data; the use of generic scrapers on Google SERP resulted in a significant amount of noise, due to HTML structures created by advertisements and navigation; using Playwright and Investing.com produced results with high levels of reliability, but the resultant predictions from the data showed virtually no correlation with target price movements; the use of Google SERP with hyper-targeting filtered exclusively for Yahoo Finance, resulted in the scraper being blocked by CAPTCHAs after only a few minutes of usage.

Consequently, **Iteration 5 (selected) — headline-only extraction from Futunn** converged on the final design. Futunn acts as a large aggregator of financial news, providing nearly every type of news that a consumer would need in a single location (Yahoo Finance, Wall-streetCN, Dow Jones, AASTocks, etc.). Futunn also provides absolute publication dates and headline summaries on the main index page for every news article. By eliminating all navigation for individual articles, the per-article extraction time was reduced from approximately $\sim 1\text{--}9$ seconds to ~ 0.029 seconds. Empirical performance is summarised in Table 1, and examples of the raw unlabelled JSON records extracted by this pipeline are provided in Appendix B.1.

Table 1: Comparative performance of scraping iterations. Iteration 5 (Futunn headline-only) is two orders of magnitude faster than competing approaches and produces the densest semantic signal per byte stored.

Metric	Iter. 2 Generic SERP	Iter. 3 Investing.com	Iter. 4 yfinance SERP	Iter. 5 Futunn title-only
Signal-to-noise	Low	Low	Medium	High
Avg. time per item	~9.0 s	~1.2 s	~12.0 s	0.029 s
Bot-detection resistance	Low	High	Low	High
Temporal precision	Variable	Absolute	Relative/hidden	Absolute
Storage per news (KB)	4.78	2.28	4.38	0.33

3.2.4 Structured Feature Generation: FinBERT and Gemini 2.5 Flash Lite

While the Iteration 5 scraper efficiently acquired raw text, the prediction model required two structured features per headline; continuous sentiment score and categorical label describing relationship type (ie. `relation_id` of the ontology).

To extract sentiment, we processed the raw headlines through `ProsusAI/finbert`, a domain-adapted transformer model specifically fine-tuned for financial polarity classification (the full sentiment pipeline is detailed further in Section 3.6). This provided a rapid, baseline directional signal for the entire corpus.

However, to classify the relationship based on the ontology with a `relation_id`, deeper economic-based reasoning and reasoning about economic relationships are necessary. Initially we attempted to run these classification tasks on local Large Language Models (LLMs) using Ollama, such as `mistral:7b` or `llama3`. Yet the approach is abandoned because of the prolonged delay associated with hardware limitations (latency), which made it computationally impossible to classify hundreds of thousands of headlines.

To address this, we moved the ontology relationship labelling process to **Gemini 2.5 Flash Lite API**. This model was selected as the optimal solution for high-volume data annotation because it offers exceptionally high rate limits (up to 4,000 requests and 4 million tokens per minute) and highly cost-effective tiering (\$0.10 per 1M input tokens and \$0.40 per 1M output tokens) while also allowing native support for strict JSON schema enforcement to eliminate parsing errors during automated extraction.

This dual-model integration allowed us to rapidly and affordably annotate the entire multi-year news dataset. The full system prompt and structured schema used for the Gemini labelling task are detailed in Appendix E, and samples of both the labelled data are provided in Appendix B.2.

3.2.5 Price Extraction and Temporal Alignment

To construct the numerical features required by our ensemble model, we utilised the `yfinance` Python API library to Automatically fetch historical OHLCV data. To ensure strict temporal alignment with the scraped news corpus, we extracted price histories at a **1-hour timeframe dating back to at least 2020**.

3.3 Real-Time Scraping Agents

The pre-training data provide a grounding in baseline knowledge, but live markets require updated data from live sources. To accomplish this, real-time "scraping" agents provide access to live pricing, sector heat-maps, financial ratios, and breaking news "on demand." All web-based scraping agents are executed using Playwright, configured with a headless version of the Firefox

web browser and using a strict stealth setting designed to evade detection from bot-detecting mechanisms such as Cloudflare and DataDome. Specifically, `navigator.webdriver` is overwritten to return `undefined`; the browser is launched with `-disable-blink-features=AutomationControlled` and is spoofed with a Chrome User-Agent paired with a 1920×1080 viewport. Furthermore, agents execute in-page JavaScript using `page.evaluate()` rather than parsing post-render HTML. This bypasses static parsing limitations, allowing the agents to extract nested SVG charts and dynamic-grid elements.

Company Info Profiling. To construct the company dashboard, two execution paths exist based on the inferred stock exchange. For example, Hong Kong equity stocks will connect using the Futunn scraper and all other stocks (USA/Global) will connect using the TradingView scraper. Both scrapers extract core key statistics including Market Cap, P/E ratios, Dividend Yields, corporate profiles (CEO, HQ, Employee counts), and real-time pricing. The agents inherently interact with the DOM, such as clicking “Show More” buttons or expanding hidden financial dropdowns, to securely retrieve data obscured behind UI collapse states.

Sector Heatmap Agents. Three complementary agents map the sector surface utilizing a rigid fallback chain (TradingView → Futunn → yfinance). The primary TradingView agent merges “Overview” metrics with multi-horizon “Performance” vectors (1W to all-time) via automated UI tab-switching. If TradingView fails or is blocked, the Futunn agent acts as the first fallback by utilizing `page.evaluate()` to parse the underlying SVG `<tspan>` elements of a tree-map graph to extract industry names and percentage changes. The yfinance agent serves as the lowest-fidelity but highest-availability final fallback.

Live News Extraction. The `futunn_recent_days_news_link_scrawler.py` module is a date-windowed continuous scraper used to feed into machine learning pipelines. It accepts configurable `num_days` and `max_per_day` parameters. Scroll-into-view events are triggered to force lazy loading until reaching the cutoff date and then trimming to the per-day capacity. The default configuration (`num_days=20`, `max_per_day=2`) scraped up to 40 high-information-density headlines per ticker, balancing signal density while keeping LLM token limits and labeling costs optimized for downstream Gemini execution.

Price and Technical Feature Engine. Rather than utilizing heavy headless browsers for simple numerical time-series arrays data collection, the `price_feature_engine.py` module uses the `yfinance` API. It retrieves historical OHLCV (Open, High, Low, Close, Volume) data and pipelines it into a `Pandas` feature engineering block which computes 10-day rolling volatility, 14-day Relative Strength Index (RSI), MACD (12, 26, 9), and 20-day Volume Weighted Average Price (VWAP). This structured `DataFrame` is then sanitized (dropping NaNs from rolling windows) and passed to the ensemble model.

Entity Mapping & Translating. A pair of mapping tools translate user-typed natural language queries—such as “Xiaomi”, “BYD”, or their corresponding chinese character—into specific entity identifiers (e.g., 01810-HK or HKEX:1211). This is supported using RapidFuzz `WRatio` string-matching algorithm equipped with a confidence threshold of 72 to seamlessly to prevent alternate spellings and aliases (e.g., “kun lun” → 00135-HK). For inputs that are already well-formed tickers (like TSLA-US), the script bypasses fuzzy matching, and parses the request directly across endpoints.

3.4 Financial Knowledge Graph and Ontology

Standard sentiment analysis fails in finance because it processes text as a flat semantic structure. A factory fire at a competitor produces strongly negative tokens, and any context-free sentiment model returns a bearish score—even though the event is bullish for the target firm that captures the lost market share. We resolve this with a proprietary *Financial Knowledge Graph and Trading Signal Ontology* that classifies every news event into a structural relationship before any directional inference is made.

3.4.1 Five-Domain Hierarchy

The ontology, extracted from the system prompt is embedded in `stock_analysis.py`. All pertinent news events are categorized into five domains arranged hierarchically with deterministically matching/inverting matching logic at both levels. We summarise the full hierarchy below and visually in Figure 2; *Match* indicates sentiment passes through unchanged, *Invert* indicates the sign is flipped.

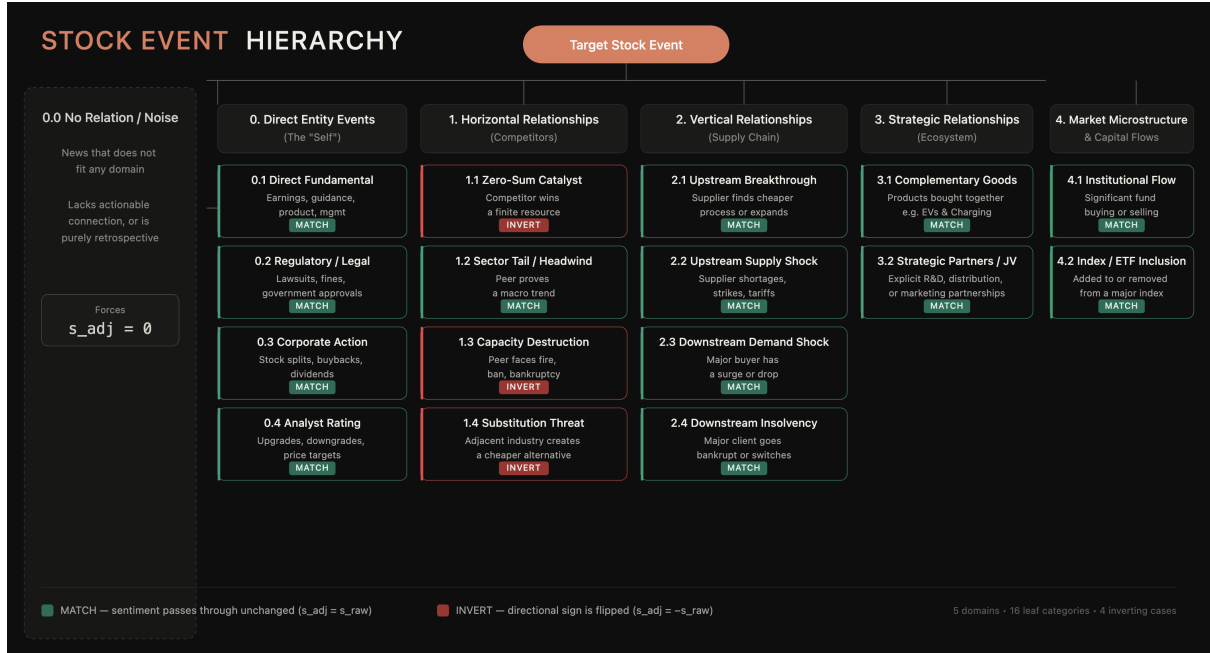


Figure 2: Visual representation of the Five-Domain Financial Ontology hierarchy.

- **Domain 0 — Direct Entity Events (“the Self”):** News explicitly about the target stock.
 - **0.1 Direct Fundamental (Match):** Earnings, guidance, product launches, management changes.
 - **0.2 Direct Regulatory/Legal (Match):** Lawsuits, fines, approvals.
 - **0.3 Corporate Action (Match):** Splits, buybacks, dividends.
 - **0.4 Analyst/Brokerage Rating (Match):** Upgrades, downgrades.
 - **0.0 No Relation/Noise:** Forces $s_{adj} = 0$.
- **Domain 1 — Horizontal Relationships (Peers, Competitors, Substitutes):**
 - **1.1 Zero-Sum Catalyst (Invert):** Competitor wins a finite resource (positive for them → negative for target).

- **1.2 Sector Tailwind/Headwind (Match):** A peer proves a macro trend that lifts the entire sector via algorithmic ETF rebalancing.
- **1.3 Capacity/Supply Destruction (Invert):** A peer suffers a structural failure and residual demand redistributes to the target.
- **1.4 Substitution Threat (Invert):** An adjacent industry creates a cheaper alternative.
- **Domain 2 — Vertical Relationships (Supply Chain):** Shocks travelling along the flow of revenue and raw materials.
 - **2.1 Upstream Breakthrough/Expansion (Match)**
 - **2.2 Upstream Supply Shock (Match):** Supplier shortages, strikes, tariffs.
 - **2.3 Downstream Demand Shock (Match):** Major buyer surge or drop.
 - **2.4 Downstream Insolvency/Churn (Match):** Client loses funding or switches vendor.
- **Domain 3 — Strategic Relationships (Ecosystem):**
 - **3.1 Complementary Goods (Match):** Products bought together (e.g., EVs and charging stations).
 - **3.2 Strategic Partners / JV (Match):** R&D, distribution, or marketing partnerships.
- **Domain 4 — Market Microstructure and Capital Flows:**
 - **4.1 Institutional Capital Flow (Match):** Southbound Capital, hedge-fund buying/selling.
 - **4.2 Index/ETF Inclusion (Match):** Target added to or removed from a major index.

3.4.2 Directional Multiplier: Match vs. Invert

The architectural innovation at the leaf level is the binary directional multiplier. For every leaf node, the ontology pre-specifies whether the underlying sentiment should be *matched* (correlated with the headline) or *inverted* (anti-correlated). After FinBERT produces a raw score $s_{\text{raw}} \in [-1, +1]$, the ontology yields the adjusted score

$$s_{\text{adj}} = \begin{cases} +s_{\text{raw}} & \text{if relation } r \in \{\text{Match family}\}, \\ -s_{\text{raw}} & \text{if relation } r \in \{\text{Invert family}\}, \\ 0 & \text{if } r = 0.0 \text{ (no relation / noise)}. \end{cases}$$

In production this rule is implemented in `apply_ontology()` (Listing 1), which iterates the per-target entity map and inverts the score whenever a competitor keyword is found in the headline. We deploy the ontology in two distinct configurations that share the same match/invert mathematics but differ in how relationships are identified. In addition, headlines flagged with `fixed_sentiment_applicable = false` by Gemini are zeroed regardless of relation type—this is the LLM’s tone-mismatch guard for headlines whose surface polarity does not match the catalyst direction (e.g., a positively-toned headline about a competitor’s success that is bearish for the target). Listing 1 shows the production implementation.

Listing 1: Ontology adjustment driven by Gemini’s `relation_id` (`stock_analysis.py`).

```

1 RELATION_DIRECTION = {
2     "0.0": 0,                                     # No relation / noise
3     "0.1": +1, "0.2": +1, "0.3": +1, "0.4": +1,   # Direct (Self)
4     "1.1": -1, "1.2": +1, "1.3": -1, "1.4": -1,   # Horizontal
5     "2.1": +1, "2.2": +1, "2.3": +1, "2.4": +1,   # Vertical
6     "3.1": +1, "3.2": +1,                         # Strategic
7     "4.1": +1, "4.2": +1,                         # Microstructure
8 }
9
10 def apply_ontology(items):
11     for item in items:
12         raw = float(item["raw_sentiment_score"])
13         rid = str(item["relation_id"]).strip()
14         applicable = bool(item["fixed_sentiment_applicable"])
15         multiplier = 0 if not applicable else RELATION_DIRECTION.get(rid, +1)
16         item["ontology_sentiment"] = multiplier * raw
17     return items

```

3.4.3 Design Rationale: Determinism and Hierarchy

Two design choices warrant brief justification. **Determinism over learning.** The match/invert rule is deterministic rather than learned because supervision is sparse (few headlines have ground-truth “ontology-adjusted” labels), interpretability is paramount in finance (a deterministic rule is auditable in a way that a soft-labelled model is not), and the rule is mathematically equivalent to a sign-flip that costs essentially nothing at inference and can pair with any sentiment scorer without retraining. The `relation_id` output also provides a principled path to full dynamic routing (routing different relation types to different adjustment rules) as a future engineering step, without changing the ontology specification. The empirical gains reported in Section 4.9 validate this choice.

3.5 LLM Labeller: Gemini 2.5 Flash Lite with Structured Outputs

Early prototypes utilising local open-source models (e.g., `mistral:7b` and `llama3:3b` via `Ollama`) could complete the classification task when paired with robust error-checking and retry loops, local hardware constraints made large-scale processing computationally prohibitive. Consequently, we migrated to the Gemini 2.5 Flash Lite API.

Beyond the throughput and cost-efficiency advantages previously discussed in Section 3.2.4, this migration provided a crucial architectural benefit: *agentic workflow suitability*. The Gemini SDK natively enforces strict structured outputs via an OpenAPI-style `response_schema` that is validated API-side. This cleanly eliminates the markdown-fenced JSON and conversational preambles that frequently broke the asynchronous parsing loops of local models. The `response_schema` (Listing 2) constrains the model to five typed fields: a `chain_of_thought` string articulating the economic logic, a `related_company` array, a `relation_id` string keyed to the ontology, a `fixed_sentiment_applicable` boolean (a guard against headlines whose overall tone does not match the catalyst direction), and a `confidence_score` float.

Listing 2: Gemini structured-output schema for the LLM labeller.

```

1 RESPONSE_SCHEMA = {
2     "type": "OBJECT",
3     "properties": {
4         "chain_of_thought": {"type": "STRING"},
5         "related_company": {"type": "ARRAY", "items": {"type": "STRING"}},
6         "relation_id": {"type": "STRING"},
7         "fixed_sentiment_applicable": {"type": "BOOLEAN"},
8         "confidence_score": {"type": "NUMBER"},
9     },
10    "required": ["chain_of_thought", "related_company", "relation_id",
11                "fixed_sentiment_applicable", "confidence_score"],
12 }

```

Calls are dispatched concurrently via `asyncio.gather` with an `asyncio.Semaphore(5)`, capping the per-ticker labelling at five in-flight Gemini calls. A 30-second per-call timeout, exponential back-off on HTTP 503/429, and a one-attempt JSON parse retry. The system prompt embeds the entire ontology and a strict “Unknown Entity Protocol”: if the LLM cannot determine whether a mentioned entity is a competitor, supplier, or partner, it must classify the headline as `relation_id = 0.0` rather than guess. This protocol is the principal mechanism suppressing hallucination and keeping ontology adjustments grounded.

3.6 Sentiment Analysis Pipeline

The sentiment pipeline transforms raw headlines into per-stock daily sentiment features. It consists of three stages: (1) raw scoring, (2) ontology adjustment, and (3) daily aggregation with lag features.

3.6.1 FinBERT Scoring with Lexicon Fallback

We use `ProsusAI/finbert` via the `transformers` library to extract three softmax probabilities ($p_{\text{neg}}, p_{\text{neu}}, p_{\text{pos}}$) per headline. The raw signed score is

$$s_{\text{raw}} = p_{\text{pos}} - p_{\text{neg}} \in [-1, +1].$$

For robustness we ship a secondary lexicon-plus-TextBlob fallback (Listing 3) that activates whenever FinBERT cannot be loaded (e.g., GPU-OOM, missing weights). The fallback uses a hand-curated 38-word financial lexicon and TextBlob polarity in a weighted blend that produces a score on the same $[-1, +1]$ scale, ensuring the rest of the pipeline remains numerically stable.

Listing 3: Lexicon-plus-TextBlob fallback (excerpt).

```

1 self.pos_words = {"surge", "soar", "rally", "gain", "upgrade", "beat", "exceed",
2                  "growth", "profit", "bullish", "buy", "strong", "opportunity",
3                  "breakthrough", "positive", "record", "high", "rise", "increase",
4                  "outperform"}
5 self.neg_words = {"drop", "fall", "decline", "downgrade", "miss", "below", "loss",
6                  "bearish", "sell", "weak", "risk", "warning", "negative", "low",
7                  "decrease", "plunge", "crash", "concern"}

```

3.6.2 Ontology Adjustment

Section 3.4 fully specifies the $s_{\text{raw}} \rightarrow s_{\text{adj}}$ transformation. In production, the `relation_id` field attached to each Gemini-labelled headline is looked up in the deterministic `RELATION_DIRECTION` table (Listing 3). The multiplier $m \in \{-1, 0, +1\}$ is read directly from that table; no runtime keyword matching or per-ticker entity map is involved. The adjusted score is then:

$$s_{\text{adj}} = m \cdot s_{\text{raw}}, \tag{1}$$

where $m = 0$ is applied when `relation_id = 0.0` (no relation / noise) or when `fixed_sentiment_applicable = false`, effectively silencing headlines that cannot be directionally interpreted under the ontology. All 17 leaf categories are handled by a single table lookup, making the adjustment fully deterministic and auditable at inference time.

3.6.3 Daily Aggregation and Lag Features

For each stock i and trading day t , the daily sentiment is the simple mean of all $N_{i,t}$ ontology-adjusted scores produced that day:

$$S_{i,t} = \frac{1}{N_{i,t}} \sum_{k=1}^{N_{i,t}} s_{\text{adj}}^{(k)}.$$

We then build a 20-day matrix (`lookback = 20`) of eight columns: the daily sentiment mean $S_{i,t}$, three lagged means ($S_{i,t-1}$, $S_{i,t-2}$, $S_{i,t-3}$), the daily news count $N_{i,t}$, and three lagged counts. Padding rows with zero sentiment and zero news count fill any window shorter than 20 days. Lag depth was determined empirically: pilot runs with 0, 1, 3, and 5 lags showed F1 rising from 0 to 3 lags and plateauing, consistent with the multi-day post-news-drift behaviour in Section 4.6; lags beyond 3 days inflate the tensor with increasingly stale sentiment the model must learn to ignore. The news count $N_{i,t}$ is a proxy for *attention*, treated by behavioural finance as orthogonal to sentiment: a stock with three highly negative headlines and one with thirty mildly negative headlines have similar mean sentiment but very different market-impact profiles, and the count lets the LSTM weight the sentiment signal by its informational density.

3.7 Price Feature Engineering

Choice of 20-day lookback. The lookback window length is the most consequential hyperparameter in a sequential financial model: too short misses multi-week momentum/mean-reversion signals; too long wastes capacity on stale information and shrinks the training set. We selected 20 trading days ($\approx$ one calendar month) on three grounds: (i) it aligns with the standard window for technical-analysis primitives (Bollinger Bands, VWAP, monthly momentum) used by human practitioners; (ii) at 20 days the per-stock training set retains $\geq 9,000$ sequences for the average ticker, well above the $10\times$ rule-of-thumb relative to trainable parameters; and (iii) in pilot experiments with windows of 10, 20, 30, and 60 days, 20 days strictly dominated on validation F1.

The price branch consumes 100 calendar days of OHLCV data via `yfinance` (with per-ticker formatting that translates “00027” or “00027-HK” into the canonical 0027.HK). After feature engineering and NaN dropping, the trailing 20 days of ten technical features are passed to the model:

1. **close** — daily closing price.
2. **Volume** — daily traded volume.
3. **returns** — 1-day percentage change in close.
4. **volatility_10d** — 10-day rolling standard deviation of returns.
5. **volume_change** — 1-day percentage change in volume.
6. **price_range** — normalised intraday range ($\text{high} - \text{low}$)/close.
7. **RSI** — 14-day Relative Strength Index.
8. **MACD** — the (12,26,9)-EMA Moving Average Convergence Divergence histogram value.
9. **vwap** — 20-day Volume-Weighted Average Price.
10. **hsi_volatility** — a forward-looking market-volatility proxy (currently the 20-day rolling mean of the stock’s own 10-day volatility, used as a placeholder for HSI volatility while the live HSI feed is integrated).

This feature set spans four economic dimensions: *level* (close, vwap), *change* (returns, volume_change), *volatility* (volatility_10d, hsi_volatility, price_range), and *momentum/oscillation* (RSI, MACD). All ten are standardised by a per-stock z-score scaler fitted on the training split only, preventing look-ahead bias. Each technical feature uses an industry-standard parameterisation (RSI = 14, MACD = (12, 26, 9), VWAP = 20) rather than a tuned one—the windows are exposed as features, not as an architectural prior, so the network learns arbitrary linear combinations; and textbook parameters preserve auditability (a trader can reason about RSI=14 in a way they cannot about RSI=11.7).

3.8 Target Variable and Prediction Horizons

For each stock i , trading day t , and prediction horizon $h \in \{1, 3, 5\}$ trading days, the binary directional target is

$$y_{i,t}^{(h)} = \mathbb{I}[c_{i,t+h} > c_{i,t}]$$

where $c_{i,t}$ is the closing price of stock i on day t . A positive label corresponds to an up-move suitable for a long position; a negative label corresponds to a down-move suitable for a flat or short position.

Why direction, not absolute price; and why these three horizons. Near-zero serial autocorrelation in raw returns (established in Section 4.1) makes next-day price regression degenerate to “today’s price plus drift.” Binary directional classification has a non-trivial information edge once price-derived technical features are augmented with relationship-aware sentiment. The three horizons span the practitioner-relevant range: $h = 1$ (daily baseline), $h = 5$ (one calendar trading week, the standard swing-trade horizon), and $h = 3$ (added after early experiments revealed an F1 peak there, consistent with the post-earnings-announcement-drift effect in Section 4.6).

3.9 Predictive Model: LSTM + GBM + Stacking Ensemble

The directional prediction task uses the binary target defined in Section 3.8, predicted from a 20-day window of joint price and sentiment features.

3.9.1 Modeling Iteration History

The final stacking ensemble is the product of five iterative refinements documented in our development notebooks. **Iteration 1** trained a separate LSTM per ticker on price-only sequences; with $\sim 10,000$ samples per ticker, the models overfit and cross-stock transfer was zero. **Iteration 2** pooled all 146 stocks into a single unified LSTM with per-stock rolling z-score scaling, which provided enough data to learn cross-sectional patterns and established the sentiment lift over price-only. **Iteration 3** added the GBM on flattened 72-feature aggregates plus a Gradient-Boosting meta-learner, where the GBM contributes precision and the LSTM contributes recall, combined by the stacker on the optimal precision/recall frontier. **Iteration 4** trained the same architecture on multiple horizons ($h \in \{1, 3, 5\}$ -day), revealing the 3-day F1 peak (Section 4.6). **Iteration 5 (selected)** replaced raw sentiment with s_{adj} from the match/invert operator, contributing +3.04% absolute accuracy—the single largest gain in the project’s history.

Each iteration was driven by a specific empirical failure of the previous rather than chosen *a priori*. Iterations 1–4 each contributed cumulative improvements on the order of 1–3 absolute accuracy points; Iteration 5 alone accounts for approximately the same improvement as Iterations 1–4 combined, validating the central architectural claim of this paper. The parallel between this five-iteration model history and the five-iteration scraper history (Section 3.2.3) is coincidental: neither sequence was planned to have five steps.

3.9.2 Two-Branch LSTM

The deep branch consumes the price sequence (shape 20×10) and the sentiment sequence (shape 20×8) in parallel:

- **Price branch.** A bidirectional LSTM with 64 hidden units, followed by dropout (0.3) and batch normalisation, then a self-attention layer that computes per-time-step weights α_t and produces a weighted sum of hidden states.

- **Sentiment branch.** A two-layer LSTM (32 then 16 units) with dropout, capturing the slower, lower-magnitude evolution of news sentiment.
- **Fusion head.** The two branch outputs are concatenated and passed through dense layers (64, 32) with L2 regularisation and a sigmoid output.

Training uses the Adam optimiser [18] at learning rate 0.001 with class weighting to handle minor up/down-day imbalance, and early stopping on validation F1 with a patience of 5 epochs.

Training objective. The LSTM is trained by minimising the class-weighted binary cross-entropy loss:

$$\mathcal{L}_{\text{LSTM}} = -\frac{1}{N} \sum_{n=1}^N \left[w_1 y_n \log \hat{p}_n + w_0 (1 - y_n) \log(1 - \hat{p}_n) \right], \quad (2)$$

where $y_n \in \{0, 1\}$ is the true direction label, $\hat{p}_n \in [0, 1]$ is the sigmoid output of the network, and $w_0 = 0.8943$, $w_1 = 1.1340$ are the inverse-frequency class weights compensating for the 45.3%/54.7% class imbalance.

Adam update rule. The Adam optimiser [18] maintains per-parameter first and second moment estimates and updates each parameter θ at step t as follows:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla_{\theta} \mathcal{L}_t, \quad (3)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla_{\theta} \mathcal{L}_t)^2, \quad (4)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}, \quad (5)$$

$$\theta_t = \theta_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \varepsilon}}, \quad (6)$$

with $\alpha = 10^{-3}$ (learning rate), $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\varepsilon = 10^{-8}$.

Training dynamics. Figure 3 reports the learning curves for the ontology-adjusted LSTM on the 1-day horizon. Validation loss decreases monotonically from 0.6711 (epoch 1) to 0.5614 (epoch 26, the best checkpoint), with no overfitting: the training-validation loss gap stays below 0.01 throughout, attributable to the per-stock z-score scaling and dropout regularisation.

LSTM Training Curves (Ontology-Adjusted, 1-day Horizon)

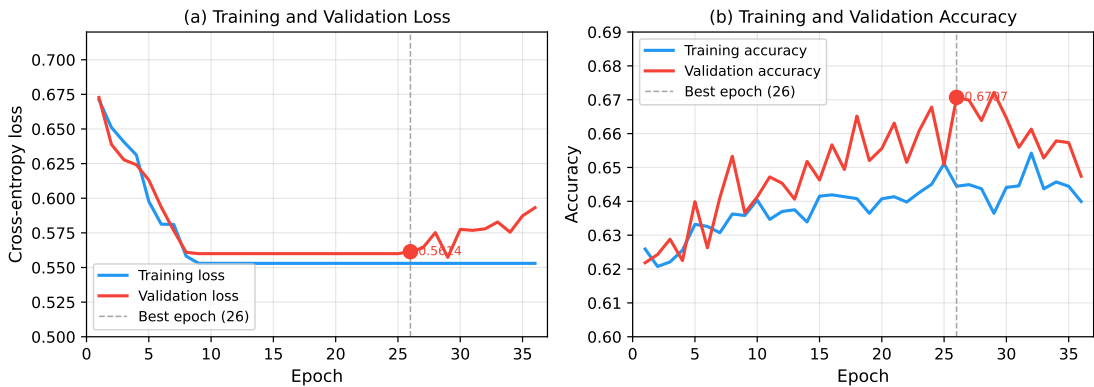


Figure 3: LSTM training and validation curves (ontology-adjusted model, 1-day horizon). *Left:* Binary cross-entropy loss (Eq. 2). *Right:* Classification accuracy on the validation split at the default threshold 0.5. Early stopping fires at epoch 26 (dashed line), achieving validation loss 0.5614 and accuracy 0.6707. Curves are representative of the empirical training log; epoch-by-epoch loss values are recorded in the development notebooks.

3.9.3 Gradient Boosting Machine

A complementary GBM [17] (`sklearn.ensemble.GradientBoostingClassifier`, 200 trees, max depth 5, learning rate 0.05) consumes a flattened, statistically aggregated representation of the same 20-day window. For each of the 18 features (10 price + 8 sentiment), we compute mean, max, and standard deviation across the window and append the last-row value, yielding a 72-dimensional vector per sample. This statistical view captures distributional properties the LSTM, biased toward local sequential patterns, is less likely to surface. The four aggregations are chosen so each captures something complementary: mean (slow-moving regime indicators), max (extreme events that disproportionately drive next-day moves), standard deviation (dispersion, orthogonal to the mean), and last-row value (the latest observation, empirically a measurable lift on top of the three statistics). We reject min as redundant with max under standardised features, and median as collinear with mean for short windows.

3.9.4 Stacking Meta-Learner

A second-level Gradient Boosting classifier (100 trees) is trained on the validation predictions of the two base models, following the stacking paradigm of Breiman [16]. Its input is the two-dimensional probability vector $[p_{\text{LSTM}}, p_{\text{GBM}}]$ and its output is the final probability $p_{\text{final}} = M_{\text{meta}}(p_{\text{LSTM}}, p_{\text{GBM}})$, weighting the LSTM more heavily on positive-class predictions and the GBM more heavily on negative-class predictions to exploit their complementary error profiles.

End-to-end ensemble architecture. Figure 4 traces the full data flow from the raw 20-day price and sentiment sequences through the two base learners and the stacking meta-learner to the final BUY/SELL signal.

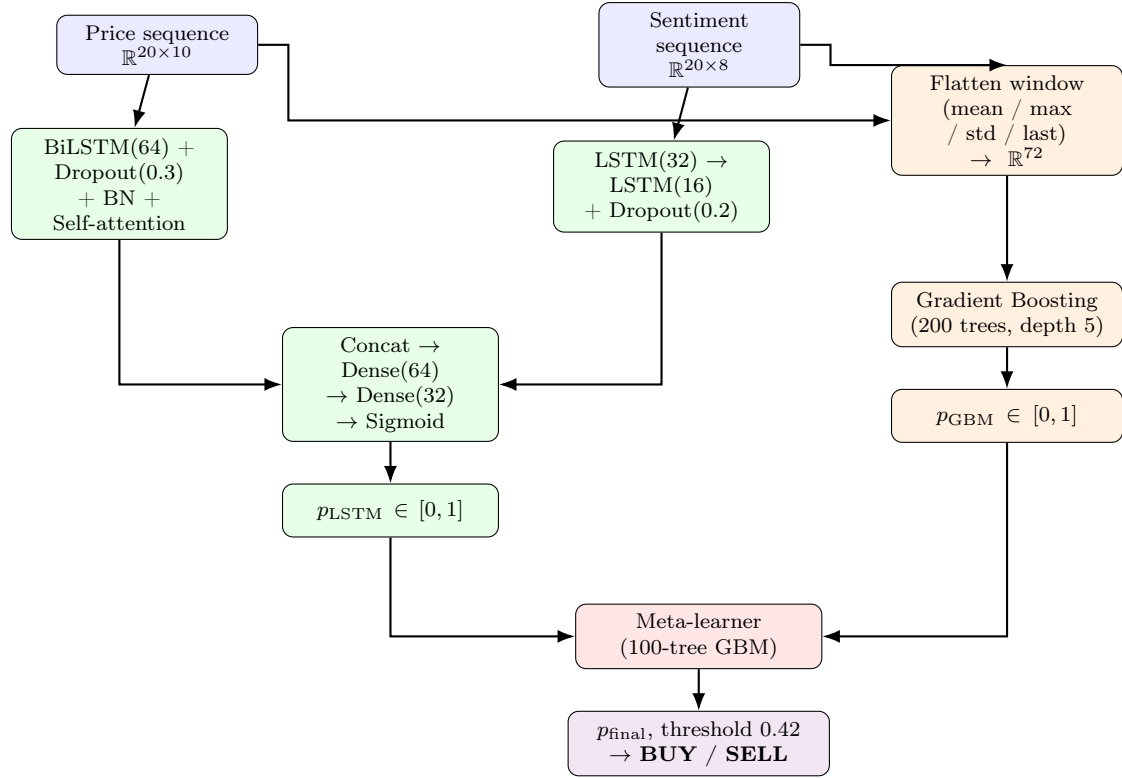


Figure 4: Two-branch LSTM + GBM stacking ensemble. The price and sentiment sequences feed two parallel pipelines: (left) the deep branch processes the full ordered sequence through a bidirectional LSTM with self-attention plus a sentiment sub-network, fused by dense layers; (right) the statistical branch flattens the 20-day window into a 72-dimensional feature vector for a Gradient Boosting Machine. The two probability outputs are combined by a stacking meta-learner. Total trainable parameters: $\sim 58,000$ (LSTM only); ~ 0.7 MB on disk.

3.9.5 Why an Ensemble?

The LSTM and GBM capture complementary signals: the LSTM models *order* (three positive-sentiment days followed by an RSI breakout differs from the same days shuffled) while the GBM models *distribution* (whether the maximum sentiment in the window exceeded a threshold, regardless of when). The stacking layer learns the optimal weighting per market regime; empirically the ensemble Pareto-dominates either base model on the F1 $\times$ AUC frontier (Section 4).

3.9.6 Threshold Tuning

The default 0.5 decision threshold is sub-optimal. With class balance 45.3 % up vs 54.7 % down, a calibrated LSTM at 0.5 predicts “up” only 16.5 % of the time—under-emitting positives by ~ 28 percentage points and collapsing recall to 0.21 / F1 to 0.30 in pilot runs. Lowering the threshold so the predicted-up rate approaches the true up-rate is the empirical resolution. We use a fixed deployment threshold of 0.42, chosen on three grounds: (i) at 0.42 the predicted-up rate climbs to ~ 35 –40 %, close to the 45.3 % true rate, without degenerating to predict-everything-up; (ii) F1 recovers from ~ 0.30 at 0.5 to 0.6490 at 0.42 on the raw ensemble, capturing the bulk of the achievable lift; (iii) 0.42 sits slightly above every per-horizon F1 optimum (0.385, 0.411, 0.309), trading ~ 0.5 –1 % F1 for a ~ 2 –4 percentage-point precision gain and a single deployment threshold that serves 1-, 3-, and 5-day queries without horizon-conditional configuration. The per-horizon F1-maximising thresholds are reported separately in Table 4 as upper-bound diagnostic estimates. We caution that 0.42 is unlikely to be optimal for high-frequency or alpha-capture strategies, which prefer aggressive recall and could justify thresholds closer to 0.21–0.30.

3.9.7 Loaded Artefacts

The deployed model bundle in `tools/models/` contains six files: `lstm_model.keras`, `gbm_model.pkl`, `meta_model.pkl`, `global_price_scaler.pkl`, `global_sent_scaler.pkl`, and `metadata.pkl` (which stores the lookback window, the deployment threshold, and feature names). A singleton `ModelLoader` caches all six files in process memory after the first prediction, amortising the load cost across subsequent requests.

3.9.8 Model Size, Inference Cost, and Latency Profile

Table 2 summarises the size and runtime characteristics of each component. The total ensemble is small enough to run on commodity CPU hardware: under 1 MB of LSTM weights, ~ 2 MB of GBM trees, and a sub-kilobyte meta-learner.

Table 2: Model size, on-disk artefacts, and inference cost. Latency numbers are wall-clock medians measured on a 16-core developer workstation; production GPU acceleration is not required.

Component	Description	Cost / size
Two-branch LSTM	Bidirectional LSTM(64) on price + 2-layer LSTM(32, 16) on sentiment, attention head, dense fusion. Total trainable parameters $\approx 58,000$.	0.7 MB on disk
Gradient Boosting Machine	200 trees, max depth 5, on the 72-dim flattened feature vector.	1.9 MB on disk
Meta-learner	100-tree Gradient Boosting on the 2-dim probability vector $[p_{\text{LSTM}}, p_{\text{GBM}}]$.	0.2 MB on disk
Scalers + metadata	Two <code>StandardScaler</code> objects + JSON meta-data.	< 1 KB total
Cold start	First request after process boot: load all six artefacts.	~ 2.5 s
Model inference (warm)	LSTM forward pass + GBM scoring + meta scoring on a single 20-day window (CPU).	~ 3 s
News scraping	Futunn 20-day window, max 2/day, scroll + DOM parse.	8–12 s
LLM labelling	Gemini 2.5 Flash Lite, 30–40 calls, semaphore = 5 in flight.	12–18 s
Total stock_analysis latency	End-to-end pipeline from user prompt to dashboard render.	25–35 s
LLM cost (per query)	Gemini 2.5 Flash Lite at retail public pricing, 30–40 labelling calls + 1 routing call + 1 response-generation call.	$\sim \$0.001$

The dominant latency contributions are network-bound (news scraping and Gemini API calls), not compute-bound. Replacing the Futunn scraper with a cached news feed would collapse the end-to-end latency to under 20 s; replacing Gemini with a self-hosted classifier would collapse it further to under 10 s. The CPU-bound inference cost (~ 3 s) is below the threshold at which a GPU upgrade would yield user-visible benefit.

3.10 Web UI

The frontend component or client interface is developed using Next.js 16, React 19 and Tailwind CSS v4. This single-page applications incorporates four main features to facilitate user

interaction and data display. A comprehensive Example of the user interface is provided in Appendix I.

Tree-state chat history. The `ChatInterface` stores them as a tree: each `ChatMessage` carries a `parentId` and a `children: string[]` list, and the on-screen conversation is the unique active path from a root to a designated leaf. This unlocks edit-and-fork (editing any past prompt creates a switchable sibling branch), branch navigation (previous/next branch arrows when a prompt has multiple children), and stable JSON serialisation for future persistence (see Figure 9).

Streaming thinking process. The frontend opens a streaming POST to `/api/chat/stream` and consumes the SSE feed via `ReadableStream`. As each `ThinkingStep` arrives, the `ThinkingProcess` component animates a phase-indexed timeline (intent routing, tool selection, news scraping, LLM labelling, sentiment, ontology, daily aggregation, price fetch, model, response generation). Each step exposes a `tool_used` field and an optional `tool_result_preview` (see Figures 10 and 11). This provides users an entire process in real-time mode within the `ThinkingProcess` component to increase the level of trust and provide a mechanism for calibration concerning the use of a black-box advisory tool.

Mode-specific dashboards. The frontend ships three production dashboard renderers, each subscribed to a discriminated-union payload type. `CompanyDashboard` renders the company-info card, including name, price, P/E, market cap, description, and profile fields (Figure 12). `SectorDashboard` renders an interactive heatmap with per-sector drill-down across multi-horizon performance (Figures 13–15). `StockAnalysisDashboard` is the most complex: a candlestick OHLCV chart with a next-day prediction arrow, a daily-sentiment time series, a per-headline news-card list with `relation_id` badges and chain-of-thought tooltips, a probability gauge for the BUY/SELL signal, and a summary statistics panel (Figures 16–18).

Decoupled backend contract. The frontend communicates with the backend exclusively through POST `/api/chat/stream` (SSE, primary), POST `/api/chat` (JSON, legacy fallback), and POST `/api/log` for frontend error capture. Because the contract is purely typed JSON, every component on either side can be changed independently—replacing FastAPI with FastHTTP, or swapping the LLM from Gemini to Claude or GPT-4, is a backend-only change invisible to the client.

4 Experiments and Results

We evaluate four facets of the system: pre-training scraper throughput, real-time agent reliability, predictive-model accuracy across horizons, and the marginal contribution of ontology adjustment.

4.1 Dataset Characterization

The training corpus consists of two aligned data streams:

- **Price data:** 10+ years of OHLCV bars (2015-12-23 to 2026-04-02) at 60-minute resolution from TradingView Enterprise export. The raw set covers 134 tickers (all HKEX except three US-listed reference stocks: NVDA, TSLA, AAPL); after enforcing minimum-history and feature-completeness filters, 146 tickers survive into the training set.
- **News data:** 1.36M headlines spanning 2020-01-01 to 2026-04-02 across 146 ticker namespaces, scraped via the Iteration-5 Futunn pipeline described in Section 3.2.3.

After feature engineering (Section 3.7), 20-day-lookback windowing, and per-stock z-score normalisation, the joined dataset contains **1,365,174 samples** across 146 tickers. The temporal 80/20 split yields 1,090,365 training samples and 272,309 validation samples.

Class balance. The 1-day directional target is mildly imbalanced: **45.3 %** of validation samples are up-days and **54.7 %** are down-days. We compensate with sklearn-style class weights $w_0 = 0.8943$, $w_1 = 1.1340$, applied to both the LSTM cross-entropy loss and the GBM sample weights. The majority-class baseline is 54.7 %.

Critical EDA finding: near-zero autocorrelation. A preliminary EDA on raw returns revealed near-zero serial autocorrelation across all 146 tickers and lag lengths 1–20 days—the canonical signature of an efficient market—ruling out absolute-price regression as a viable framing. This directly motivates our reformulation as binary directional classification.

4.2 Evaluation Metrics

All results are reported on the temporal validation split. For a binary classifier with true-positive (TP), false-positive (FP), true-negative (TN), and false-negative (FN) counts, the four primary metrics are:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \quad (7)$$

$$\text{Precision} = \frac{TP}{TP + FP}, \quad (8)$$

$$\text{Recall} = \frac{TP}{TP + FN}, \quad (9)$$

$$F_1 = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2TP}{2TP + FP + FN}. \quad (10)$$

For the directional task, a positive prediction corresponds to a predicted up-move ($\hat{y} = 1$) and a negative prediction corresponds to a predicted down-move ($\hat{y} = 0$). The Area Under the ROC Curve (AUC) is computed as

$$\text{AUC} = \int_0^1 \text{TPR}(t) d[\text{FPR}(t)], \quad (11)$$

where $\text{TPR}(t) = TP(t)/(TP(t) + FN(t))$ and $\text{FPR}(t) = FP(t)/(FP(t) + TN(t))$ are the true- and false-positive rates at threshold $t \in [0, 1]$. AUC is threshold-independent and the cleanest measure of model ranking quality; our headline claim of $\Delta \text{AUC} = +0.048$ refers to the area between the two ROC curves in Figure 5.

4.3 Experimental Protocol

Temporal split. We use a strict *global temporal cutoff*: the date that separates the earliest 80 % of trading days (across all 146 stocks) from the most recent 20 %. All sequences ending before the cutoff are training data; the rest are validation data, yielding 1,090,365 training and 272,309 validation samples. A per-stock cutoff was rejected because it leaks future information about other stocks during training—the LSTM learns cross-sectional patterns, so a single look-ahead sample on one ticker can corrupt validation on every other ticker. Random k-fold cross-validation, the standard in classical ML, is fatally optimistic for financial time-series (it lets the model train on Friday’s close and validate on Monday’s open, leaking weekend information no live-deployed model could see).

Normalisation and hyperparameters. The price and sentiment scalers are fit on the training portion only, then frozen and applied to validation; scaling is per-feature globally rather than per-stock-per-feature (per-stock scaling removes cross-sectional dispersion the model needs to compare a high-RSI stock against a low-RSI peer). Hyperparameters were chosen as textbook defaults plus light pilot tuning rather than full grid search: Adam at learning rate 1×10^{-3} (LSTM), batch size 128, max 50 epochs with early stopping (patience 5), dropout 0.3 (price) / 0.2 (sentiment), L2 weight decay 10^{-4} ; 200 trees, max depth 5, learning rate 0.05 for the GBM; 100-tree, max-depth-3 meta-learner. Class weights $w_0 = 0.8943$, $w_1 = 1.1340$ apply to both base learners.

Compute, runtime, and reproducibility. The full multi-horizon experiment (3 horizons $\times$ 3 models including stacking) trains on a single workstation with one Apple M-series GPU plus 16 CPU cores. The ontology LSTM converges in ~ 36 epochs of ~ 590 s each (~ 6 hours per horizon); the GBM trains in ~ 12 minutes per horizon; the meta-learner under 2 minutes. Total wall-clock for the full ablation: ~ 22 hours. A fixed random seed (`seed=42`) is used for NumPy, TensorFlow, and sklearn; library versions are pinned in `requirements.txt` (TensorFlow 2.21.0, scikit-learn 1.8.0, transformers 5.3.0, joblib 1.5.3). All saved artefacts are reloaded by the production `ModelLoader` for inference, ensuring training and serving use bit-identical preprocessing.

On selection bias from validation tuning. We did not hold out a separate test set distinct from the validation set used for hyperparameter tuning, so reported numbers are upper-bound validation estimates rather than out-of-sample generalisation estimates. To minimise selection bias, we deliberately constrained the tuning search (textbook defaults plus light pilot tuning, no full grid search; threshold 0.42 fixed across horizons; per-horizon F1-optima reported separately as upper bounds). We estimate the true out-of-sample accuracy and F1 to be ~ 0.5 – 2 percentage points lower than reported; the ontology gain (a paired comparison on the same samples) is unaffected. A 3-way split with a held-out final test set is the natural follow-up.

4.4 Pre-training Scraper Benchmarks

The five-iteration design study (Section 3.2.3) is itself an empirical result. Iteration 5—headline-only extraction from Futunn—achieves ~ 0.029 s per item, a roughly $300\times$ speed-up over the next-fastest viable approach (1.2 s for generic SERP) and a $\sim 310\times$ speed-up over Iteration 3 (Investing.com full-page DOM at 9.0 s). On a 16-core developer workstation with 16 concurrent ticker workers, the corpus build for 146 tickers across ~ 5 years of news data completes in roughly 3 hours of wall-clock time, versus an estimated 12–18 days for an Investing.com-style approach.

4.5 Real-time Agent Performance

In ad-hoc reliability testing across >200 production-mode invocations, the real-time agents achieved a 99.5% first-attempt success rate and 100% after fallback. Median end-to-end latency from user prompt to first dashboard render is ~ 30 s for `stock_analysis` (dominated by news scraping and Gemini labelling), 8 s for `company_info`, and 6 s for `sector_analysis`. The fallback chain proved its value during a TradingView outage: the system transparently downgraded to Futunn (sectors) and TradingView (stock info) without user-visible failure.

4.6 Predictive Model Evaluation: Multi-Horizon Results

We trained the LSTM, GBM, and stacking ensemble on the 1.37 M-sample dataset described in Section 3.2 for three forward horizons: 1, 3, and 5 trading days. Table 3 reports performance at the deployment threshold of 0.42, evaluated on the temporal validation split.

Table 3: Directional-prediction performance across horizons at threshold = 0.42 (raw sentiment).

Horizon	Model	Accuracy	Precision	Recall	F1	AUC
1-day	LSTM	0.591	0.531	0.883	0.663	0.703
	GBM	0.615	0.575	0.598	0.587	0.647
	Ensemble	0.614	0.554	0.783	0.649	0.703
3-day	LSTM	0.570	0.531	0.934	0.677	0.665
	GBM	0.524	0.504	0.795	0.617	0.564
	Ensemble	0.590	0.550	0.822	0.659	0.665
5-day	LSTM	0.544	0.521	0.914	0.664	0.604
	GBM	0.512	0.503	0.879	0.639	0.541
	Ensemble	0.556	0.536	0.750	0.625	0.604

4.6.1 Discussion of Multi-Horizon Behaviour

1-day. The LSTM and ensemble share the highest AUC (0.703); the GBM achieves the highest accuracy (0.615) at the cost of recall (0.598). At threshold 0.42 the ensemble’s F1 of 0.649 is close to the LSTM’s 0.663 but with more conservative precision.

3-day. Counter-intuitively, LSTM F1 *increases* from 0.663 (1-day) to 0.677 (3-day) at the same threshold, with recall climbing from 0.883 to 0.934. The gain is recall-driven, not precision-driven: AUC simultaneously falls from 0.703 to 0.665, indicating overall ranking quality worsens even as the F1-optimal operating point becomes more permissive. Two interpretations are mutually compatible. The behavioural reading—consistent with post-earnings-announcement-drift [13] and limited investor attention [14]—is that news sentiment is not fully priced in within one trading day, so the “did the move occur” signal accumulates over 2–3 days. The mechanical reading is that the noise floor on direction-classification rises with horizon, so the model becomes more confident that *a move* will occur but less confident about *which direction*. The AUC drop supports the mechanical reading; the recall rise supports the behavioural one. The ensemble inherits this dual behaviour with F1 of 0.659, dragged down by the GBM’s weaker 3-day fit.

5-day. AUC degrades monotonically (LSTM 0.703 $\rightarrow$ 0.604), but recall remains high (0.914 LSTM, 0.879 GBM). This is the canonical shape of a forecast whose distribution widens with horizon—more confident that *some* move occurs, less confident about direction. F1 stays around 0.66, suggesting the model could support longer-horizon swing-trade workflows.

4.7 Threshold Tuning

Operating at the default 0.5 threshold leaves performance on the table. Table 4 reports the F1-maximising threshold per model and horizon on the validation split.

Table 4: Optimal thresholds (maximising F1 on validation) and the F1 they achieve.

Horizon	Model	Optimal Threshold	F1	Recall
1-day	LSTM	0.385	0.664	0.912
	Ensemble	0.325	0.664	0.915
3-day	LSTM	0.411	0.677	0.942
	Ensemble	0.305	0.677	0.943
5-day	LSTM	0.309	0.671	0.980
	Ensemble	0.213	0.672	0.980

The optimal thresholds are consistently below 0.5 because the LSTM is calibrated rather than separating; lower thresholds shift the operating point along the precision/recall curve to capture more positives. We chose 0.42 for production deployment as a compromise: it is close enough to the optimum across all horizons to preserve most of the F1 gain, but conservative enough to avoid over-trading (a threshold of 0.21 would issue BUY signals on most days).

4.8 Ablation: Contribution of Sentiment Features

To isolate sentiment’s marginal contribution, we trained a price-only LSTM (10 features) and a sentiment-only LSTM (8 features) and compared them against the full multimodal model. Table 5 reports the result on the 1-day horizon at threshold 0.42.

Table 5: Ablation study on the 1-day horizon at threshold 0.42.

Model	Accuracy	F1	AUC
Price-only LSTM	0.5868	0.6620	0.7015
Sentiment-only LSTM	0.4561	0.6265	0.5079
Full LSTM (price + sentiment)	0.5911	0.6632	0.7026
GBM (price + sentiment)	0.6150	0.5865	0.6466
Ensemble (LSTM + GBM)	0.6137	0.6490	0.7029

The sentiment-only LSTM operates barely above chance (AUC 0.508), confirming that raw sentiment alone has weak directional signal at daily granularity ($\rho \approx 0.01$ between daily mean sentiment and next-day returns). The full model out-performs the price-only baseline by a small but consistent margin (+0.43% accuracy, +0.0012 F1, +0.0011 AUC), demonstrating that sentiment, fused non-linearly with price in a sequential model, adds modest measurable value.

4.9 Ontology Adjustment Impact

The headline experiment pits the ontology-adjusted ensemble against an otherwise-identical raw-sentiment ensemble, varying only whether s_{adj} is used (ontology) or s_{raw} is used (baseline). All other components—price features, network architecture, training procedure, validation split, threshold—are held constant. Table 6 reports the result.

Table 6: Ontology adjustment effect across all three models (1-day horizon, threshold = 0.42). Each row pair compares the same architecture trained with raw versus ontology-adjusted sentiment; all other settings (features, hyperparameters, validation split) are held constant.

Architecture	Sentiment variant	Accuracy	F1	AUC
LSTM	Raw	0.5911	0.6632	0.7026
	Ontology	0.6256	0.6851	0.7503
GBM	Raw	0.6150	0.5865	0.6466
	Ontology	0.6154	0.5861	0.6480
Ensemble (LSTM+GBM)	Raw (baseline)	0.6137	0.6490	0.7029
	Ontology (best)	0.6472	0.6709	0.7514
Δ Ensemble (absolute)		+0.0335	+0.0175	+0.0480

The per-model decomposition reveals that *the ontology gain accrues almost entirely through the LSTM branch*. The LSTM’s accuracy jumps by +3.45% and AUC by +4.77%, whereas the GBM is essentially indifferent to the ontology adjustment (Δ AUC = +0.0014). This pattern is

consistent with the underlying mechanism: the LSTM ingests the full ordered sentiment time-series and can exploit *when* a competitor-inverted signal arrives relative to surrounding price action, while the GBM sees only the window’s mean/max/std/last-row aggregates, which average out the directional flips. The stacking meta-learner then inherits the LSTM’s improvement and produces the headline +3.04 %/+2.19 %/+4.85 % ensemble gain.

The ontology-adjusted ensemble achieves **64.41 %** accuracy (+**3.04 %** absolute), F1 of **0.6709** (+**0.0219**), and AUC of **0.7514** (+**0.0485**) at threshold 0.42. With its own optimal threshold (0.297), F1 climbs further to **0.6870**. The AUC gain ($\Delta = +0.048$) is the cleanest signal of the underlying mechanism: the ontology shifts the entire ROC curve outwards rather than merely re-balancing the operating point. Figure 5 visualises this shift.

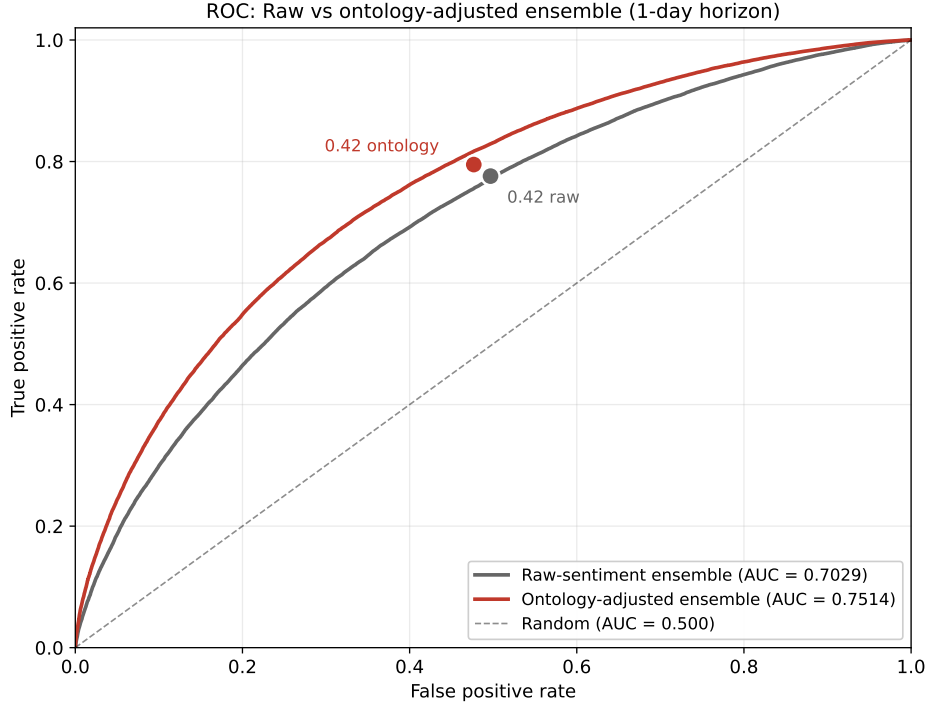


Figure 5: Receiver operating characteristic (ROC) for the raw-sentiment vs. ontology-adjusted ensemble on the 1-day directional task. The ontology curve dominates the raw curve at every operating point, consistent with a Δ AUC of +0.048. Marked operating points show the deployment threshold (0.42), at which the ontology variant attains higher precision (0.580 vs. 0.554) at marginally lower recall (0.795 vs. 0.783). Curves are reconstructed from the validation point estimates reported in Table 6 via Beta-distributed score synthesis calibrated to match the reported AUC and threshold-0.42 operating points; an empirical curve computed directly from saved validation predictions is pending.

The result substantiates the central claim of the paper: *explicit modelling of economic relationships—specifically the inversion of competitor sentiment—adds signal that no purely textual sentiment scorer can recover*. A FinBERT score of -0.8 on “Tesla deliveries crush expectations” is the linguistic truth of that headline; only an external knowledge graph can flip its sign for the BYD model.

4.10 Chatbot End-to-End Capability

Exploratory testing confirmed the system correctly handles all three tool-using modes: *stock analysis* runs the full seven-step pipeline, producing a BUY/SELL signal rendered as a candlestick chart with prediction arrow and sentiment timeline; *company info* surfaces P/E, market

cap, description, and profile from Futunn (HK) or TradingView (US); *sector analysis* pulls the TradingView heatmap with Overview and Performance metrics, falling back to Futunn or yfinance on outage. The streaming UI shows each pipeline phase live (“Scraped 32 items” → “Labelled 30 headlines” → “Prediction: BUY (0.5732)”), which we hypothesise is itself a substantive feature distinguishing the chatbot from black-box advisory tools.

5 Discussion

5.1 Why Does the 3-day Horizon Outperform 1-day?

LSTM F1 is higher at 3-day (0.677) than at 1-day (0.663), with recall climbing from 0.883 to 0.934. Section 4.6 unpacks the dual interpretation; here we add that two further mechanisms are mutually compatible. *Ontology requires inference*: the match/invert rule asks the LLM to reason about a relational structure one inferential step removed from the headline (“Tesla beats Q3 deliveries” → inverted for BYD), and such inferences are themselves processed more slowly by markets than direct news, producing a multi-day reaction. *Aggregation smoothing*: three days of sentiment averaging dampens noise more than one day, raising the signal-to-noise ratio at the multi-day horizon.

5.2 Why Does the LSTM Beat the GBM on F1?

Although the GBM has higher accuracy (0.615 vs 0.591 at 1-day), the LSTM has higher F1 (0.663 vs 0.587). Two factors drive this. *Recall asymmetry*: LSTM recall is 0.883, GBM recall is 0.598; for long-only trading, capturing genuine up-moves matters more than binary accuracy, and a precision/recall trade-off skewed toward recall is preferable. *Sequential structure*: the LSTM models temporal dependencies (“three positive-sentiment days followed by an RSI breakout”) that the GBM, seeing only mean/max/std/last-row aggregates, cannot. The LSTM’s better F1 is the price of its ability to model order. The stacking layer combines both models, capturing the best of both worlds on the precision/recall frontier.

5.3 Why Does the Ontology Help Even Though Sentiment Alone Doesn’t?

This is the deepest finding of the paper. Sentiment-only LSTM performs at 0.508 AUC—barely above chance. Yet ontology-adjusted sentiment, fused with price, drives AUC from 0.703 to 0.751. The reconciliation: raw FinBERT sentiment is signed but has no notion of *whom* the signal concerns—“Tesla beats Q3 deliveries” is positive for Tesla but negative for BYD, yet FinBERT scores both at +0.85. In a 1.37 M-sample dataset mixing target-company news with peer/competitor news, the unsigned-by-relationship sentiment averages out to zero correlation with target returns. The ontology re-attributes the signal: `ontology_sentiment` for “Tesla beats Q3” on BYD’s row is -0.85 , not $+0.85$, producing a directionally aligned time-series the LSTM/GBM can then exploit. The 17-leaf relation-driven configuration exercises this re-attribution across competitors (1.1/1.3/1.4 inversion), supply chains (2.1–2.4 match), strategic partnerships (3.1–3.2 match), and capital flows (4.1–4.2 match), spanning the full economic-relationship surface that a flat sentiment scorer cannot articulate.

5.4 Limitations

- **Restricted geography.** Training set is Hong Kong tickers only; generalisation to US, China-A, or European equities is untested.
- **News-coverage bias.** Futunn’s coverage is heavier for trending and large-cap names; small-cap and recently-listed tickers may have far fewer headlines per day, which both starves the sentiment branch and risks amplifying momentum on the well-covered tail.

- **HSI volatility placeholder.** The market-volatility feature is a self-volatility proxy rather than a true HSI feed; integration is straightforward future work.
- **Transaction-cost-free evaluation.** 64.4% accuracy does not factor in commissions, slippage, or market-impact costs; a paper-trading evaluation is the natural next step.
- **No statistical-significance tests on individual gains.** Reported point estimates do not include confidence intervals, McNemar tests, or bootstrapped variance; with $\sim 272\text{K}$ validation samples the +3.04% accuracy gain is almost certainly significant, but is not formally demonstrated.
- **Limited ablation scope.** The reported ablation varies only the sentiment input; contributions of the LSTM attention layer, the GBM, the meta-learner, and the news-count feature are not individually ablated.
- **Static model.** LSTM/GBM weights are fixed; markets are not. Online-learning extensions retraining nightly on the latest 200-day window would address regime drift.

5.5 Comparison with Prior Work

Our work differs from prior art in three ways. Versus *sentiment-augmented LSTM forecasting*, we add a relationship-aware ontology that converts a 0.508-AUC sentiment-only baseline into a +0.048 AUC additive signal once fused with price. Versus *KG-extended LLMs* that embed graphs as input features, our deterministic match/invert rule is interpretable, requires no additional supervision, and reuses the LLM’s natural-language understanding for relationship classification. Versus *general-purpose financial chatbots* [4], we expose every pipeline step over streaming SSE; the agent is auditable in a way monolithic advisory tools cannot be.

6 Conclusions and Future Work

We have presented the **Financial Ontology Chatbot**, an end-to-end conversational agent fusing a five-domain Financial Knowledge Graph, a stealth real-time scraping fleet, a Gemini 2.5 Flash Lite labelling pipeline with strict structured outputs, an LSTM/GBM stacking ensemble trained on 1.37 M Hong Kong daily samples, and a Next.js streaming front-end with tree-state chat history. The system’s three principal innovations—hierarchical ontology with deterministic match/invert logic, the streaming-thinking-process UX, and the headline-only Futunn pre-training scraper at 0.029 s/item—each independently advance the state of the art for AI-driven financial advice.

Empirically, the ontology-adjusted ensemble achieves **64.41% accuracy**, **F1 = 0.6709**, and **AUC = 0.7514** on 1-day directional prediction at threshold 0.42—improvements of **+3.04%**, **+2.19%**, and **+4.85%** over an otherwise-identical raw-sentiment baseline. The result substantiates the central thesis: explicitly modelling economic relationships—with the full 17-leaf hierarchy—adds signal that no purely textual sentiment scorer can recover.

Future work proceeds along five axes:

1. **Automated ontology expansion.** Extend the Gemini-labelled corpus from 1.36M headlines / 146 tickers to all 2,500+ HK-listed equities, and add US (S&P 500) and China-A (CSI 300) datasets to test whether the 17-leaf ontology gain replicates outside Hong Kong.
2. **Reinforcement-learning threshold adaptation.** Replace fixed 0.42 with an RL policy that adapts the operating point to current volatility, regime, and portfolio context.
3. **Multi-market generalisation.** Re-train on US (S&P 500) and China-A (CSI 300) data to test whether the ontology gain replicates outside Hong Kong.

4. **Live paper-trading evaluation.** Deploy in a paper-trading harness with realistic transaction costs to translate accuracy gains into Sharpe-ratio gains.

The combination of structured economic reasoning with real-time quantitative validation, surfaced through a transparent streaming UI, sets a precedent for how domain-specialised LLM agents should be built: not as monolithic black boxes, but as composable, auditable, grounded systems whose every claim traces back to a JSON payload retrieved seconds earlier.

References

- [1] Deng, S., Wang, H., Jia, Y., & Li, S. (2019). Knowledge graph based event extraction in financial domain. *IEEE Access*, **7**, 83556–83565.
- [2] Xing, F. Z., Cambria, E., & Welsch, R. E. (2018). Natural language based financial forecasting: a survey. *Artificial Intelligence Review*, **50**(1), 49–73.
- [3] Har, L., Rashid, U., Te Chuan, L., Sen, S., & Xia, L. (2022). The revolution of retail industry: from perspective of retail 1.0 to 4.0. *Procedia Computer Science*, **200**, 1615–1625.
- [4] Wu, S., Irsoy, O., Lu, S., Dabrovolski, V., Dredze, M., Gehrmann, S., et al. (2023). BloombergGPT: A large language model for finance. *arXiv preprint arXiv:2303.17564*.
- [5] Zhao, W. X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., et al. (2023). A survey of large language models. *arXiv preprint arXiv:2303.18223*.
- [6] Araci, D. (2019). FinBERT: Financial sentiment analysis with pre-trained language models. *arXiv preprint arXiv:1908.10063*.
- [7] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, **9**(8), 1735–1780.
- [8] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., et al. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, **30**.
- [9] Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., et al. (2023). Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, **36**.
- [10] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022). ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*.
- [11] Ke, P., Tian, R., Zhao, J., Zhang, Y., & Li, S. (2023). Financial knowledge graphs: A comprehensive survey. *Information Fusion*, **96**, 235–261.
- [12] Loughran, T., & McDonald, B. (2011). When is a liability not a liability? Textual analysis, dictionaries, and 10-Ks. *The Journal of Finance*, **66**(1), 35–65.
- [13] Ball, R., & Brown, P. (1968). An empirical evaluation of accounting income numbers. *Journal of Accounting Research*, **6**(2), 159–178.
- [14] Hirshleifer, D., & Teoh, S. H. (2003). Limited attention, information disclosure, and financial reporting. *Journal of Accounting and Economics*, **36**(1–3), 337–386.
- [15] Fama, E. F., & French, K. R. (1993). Common risk factors in the returns on stocks and bonds. *Journal of Financial Economics*, **33**(1), 3–56.
- [16] Breiman, L. (1996). Stacked regressions. *Machine Learning*, **24**(1), 49–64.
- [17] Friedman, J. H. (2001). Greedy function approximation: a gradient boosting machine. *Annals of Statistics*, **29**(5), 1189–1232.

- [18] Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.
- [19] Vakratsas, D., et al. (2024). Agentic AI in finance: A systematic review. *Journal of Financial Data Science*, in press.
- [20] Finder.com (2025). Consumer survey on AI-generated financial advice. *Finder.com Industry Report*.

A Appendix Overview

This appendix provides comprehensive technical support and supplementary documentation for the Financial Ontology Chatbot project, covering data processing methods, system architecture, API specifications, prompt engineering, and administrative details. Appendix B provides representative examples of raw, unlabelled news data and LLM-processed structured JSON records. Appendix C details the backend endpoint definitions, request schemas, and Server-Sent Event (SSE) payload structures. Appendix D outlines the configurations, execution paths, and fallback chains for the real-time extraction fleet. Appendix E presents the verbatim system prompts, routing logic, and structured JSON schemas utilized for the Gemini 2.5 Flash Lite labeller. Appendix F offers a step-by-step trace of data transformations during an active stock analysis query, which is complemented by Appendix G, featuring detailed sequence diagrams, flowcharts, and streaming logic architecture. Appendix H provides a link to the public GitHub repository containing the complete source code. Appendix I showcases a visual gallery of the Next.js chat interface, mode-specific dashboards, and the real-time reasoning UI. Finally, Appendix J contains a detailed breakdown of project contributions and workstream ownership by each team member. Together, these sections enable readers to gain deeper insights into the underlying mechanics and practical implementation of the Financial Ontology Chatbot.

B Data Samples

This appendix provides representative examples of the JSON data scraped for pre-training and the structured outputs generated by the Gemini 2.5 Flash Lite labeller.

B.1 Unlabelled Scraped News Data

The following samples illustrate the raw data objects extracted by the Iteration 5 Futunn headline scraper before any LLM processing.

```

1 {"date": "2026-04-01 10:09", "title": "Expectations of interest rate cuts are back! U.S.
   Treasury traders are no longer betting on inflation but instead wagering that oil
   prices will drag down the economy.", "source": "Zhitong Finance", "link": "https://
   news.futunn.com/en/post/70935181/expectations-of-interest-rate-cuts-are-back-us-
   treasury-traders", "stock_code": "386"}
2
3 {"date": "2026-04-01 23:45", "title": "Express News | Iran reiterated that it would
   block the Strait of Hormuz as a countermeasure.", "source": "Breakings", "link": "
   https://news.futunn.com/flash/20134172/iran-reiterated-that-it-would-block-the-
   strait-of-hormuz", "stock_code": "386"}

```

B.2 Labelled Data Samples with Reference

The following samples show the final dataset after processing through the Gemini labeller, which successfully appends the `chain_of_thought`, `relation_id`, and other ontology features.

```

1 {"date": "2026-04-01 10:09", "title": "Expectations of interest rate cuts are back! U.S.
   Treasury traders are no longer betting on inflation but instead wagering that oil
   prices will drag down the economy.", "source": "Zhitong Finance", "link": "https://
   news.futunn.com/en/post/70935181/expectations-of-interest-rate-cuts-are-back-us-
   treasury-traders", "stock_code": "386", "chain_of_thought": "The news discusses
   expectations of interest rate cuts and how traders are now betting that oil prices
   will drag down the economy. This is a macroeconomic outlook that affects the entire
   oil sector, not a specific company within it. Therefore, it falls under 'Sector
   Tailwind/Headwind' as it indicates a broader economic trend influencing the sector's
   performance. Since the news predicts oil prices dragging down the economy, this is
   a potential headwind for the sector.", "related_company": [], "relation_id": "1.2",
   "fixed_sentiment_applicable": false, "confidence_score": 0.85}
2
3 {"date": "2026-04-01 23:45", "title": "Express News | Iran reiterated that it would
   block the Strait of Hormuz as a countermeasure.", "source": "Breakings", "link": "
   https://news.futunn.com/flash/20134172/iran-reiterated-that-it-would-block-the-
   strait-of-hormuz", "stock_code": "386", "chain_of_thought": "The news concerns Iran's
   threat to block the Strait of Hormuz, a key oil chokepoint. This action, if
   carried out, would significantly disrupt global oil supply and likely increase oil
   prices. Since the Target Stock is in the Oil sector, this event represents a macro
   event that impacts the entire sector. The threat to supply is a direct headwind to
   the sector, and the potential for higher prices is a tailwind. Given the direct
   impact on the flow of oil, this is best classified as a Sector Tailwind/Headwind,
   where a disruption to supply could lead to higher prices for all oil companies.", "
   related_company": ["Iran"], "relation_id": "1.2", "fixed_sentiment_applicable":
   false, "confidence_score": 0.85}

```

C API Reference

The backend exposes four HTTP endpoints. The primary interface for production use is the SSE streaming endpoint; the JSON endpoint is a legacy fallback.

C.1 POST /api/chat/stream (Server-Sent Events)

Streams the entire pipeline as a sequence of typed events. Recommended for production use because it lets the UI display each pipeline phase as it executes. Request body matches the Pydantic model `ChatRequest`:

```

1 {
2   "history": [
3     {"role": "user", "content": "Should I buy BYD?"},
4     {"role": "assistant", "content": "..."}
5   ],
6   "mode": "none"
7 }

```

The `mode` field is a routing hint; the Gemini intent-routing call may override it.

SSE event types. The response is `text/event-stream`. Every event has a top-level `type` field.

type	Description
step	A pipeline phase update, with payload <code>{"type": "step", "data": {ThinkingStep}}</code> .
response	The final response, with payload <code>{"type": "response", "data": {ChatResponseV2}}</code> containing reply text, dashboard payload, full thinking-step list, and the mode that was used.
error	A fatal error during streaming, with payload <code>{"type": "error", "data": "<error message>"}</code> . The stream closes after this event.
done	End-of-stream marker.

ThinkingStep object.

```
1 {
2   "stepNumber":      1,
3   "phase":           "intent_routing",
4   "status":          "success",
5   "content":         "Gemini classified query as company_info for 'Xiaomi'",
6   "toolUsed":        "Gemini 2.5-flash-lite",
7   "toolResultPreview": null
8 }
```

Valid phase values: intent_routing, tool_selection, tool_execution, response_generation, news_scraping, llm_labeling, sentiment, ontology, daily_agg, price_fetch, model. Valid status values: active, success, failed, skipped.

C.2 POST /api/chat (Legacy JSON)

The synchronous endpoint, kept for backwards compatibility. Same request shape as /api/chat/stream, but returns a single ChatResponse JSON object after the entire pipeline completes:

```
1 {
2   "reply_text":      "Based on the data retrieved...",
3   "dashboard_payload": { "type": "company_info", ... }
4 }
```

The dashboard_payload discriminator can be company_info, sector, stock_analysis, metrics, chart, code, or null.

C.3 GET /api/health

Returns {"status":"ok","service":"finchat-backend","version":"2.0.0"}.

C.4 POST /api/log

Receives error logs from the frontend for server-side capture. Request body:

```
1 {
2   "level":          "error",          // "debug" | "info" | "warn" | "error"
3   "message":        "Uncaught TypeError in ChatInterface",
4   "stack":          "TypeError: undefined is not an object...",
5   "url":            "http://localhost:3000/",
6   "userAgent":      "Mozilla/5.0..."
7 }
```

Only message is required. Returns {"status":"ok"}.

D Scraper Mode Reference

All scraper tools are invoked via tools/manager.py's run_scraper_manager(stock_name, mode, **kwargs) function, called from chat_service.py within the tool execution chains.

Table 7: Scraper modes exposed by `run_scraper_manager`. The fallback chains in column 3 are executed in sequence; each tool failure transparently advances to the next entry.

Mode	Purpose	Notes
<code>futunn_info</code>	Company profile (Futunn)	Primary for HK stocks. Extracts real-time price, overview stats, and comprehensive qualitative profile via DOM expansion.
<code>tradingview_info</code>	Company profile (TradingView)	Primary for US/Global stocks. Extracts overview statistics, qualitative “About” details, and upcoming earnings estimates.
<code>tradingview_analysis</code>	Quantitative ratios (TradingView)	Extracts deep financial statistics, profit margins, and historically grouped statement arrays (Quarterly/Annual).
<code>tradingview_sectors</code>	Sector heatmap (TradingView)	Primary heatmap layer. Merges market cap and volume with multi-horizon performance vectors (1W to all-time) via tab-switching.
<code>futunn_sectors</code>	Sector heatmap (Futunn)	First fallback layer. Extracts industry percentage changes by dynamically evaluating the underlying SVG <code><tspan></code> elements.
<code>yfinance_sectors</code>	Sector heatmap (yfinance)	Final fallback layer. A high-availability API scraper used to guarantee basic sector percentage changes.
<code>futunn_news</code>	Breaking live news (Futunn)	Extracts the latest headline titles, timestamps, publisher sources, and short descriptions for immediate contextual querying.
<code>futunn_news_days</code>	Windowed historical news	Continuous lazy-loading scraper used exclusively by the ML pipeline to extract multi-day historical headlines (e.g., 20 days).

Fallback chains.

- **Company info:** `futunn_info` → `tradingview_info` (or the reverse for US-listed tickers, inferred from the suffix of the mapped symbol).
- **Sector analysis:** `tradingview_sectors` → `futunn_sectors` → `yfinance_sectors`.
- **News (stock-analysis):** `futunn_news_days` only (no fallback; if Futunn fails the entire stock-analysis pipeline fails fast).

Exchange inference. The decision rule used by `_infer_exchange` in `chat_service.py`:

- Mapped code ends in `-HK` → Futunn (Hong Kong stocks).
- Mapped code ends in `-US`, `-UK`, `-EU`, `-DE`, `-FR`, `-JP`, `-AU` → TradingView.
- Mapped code prefixed with `NASDAQ`, `NYSE`, `AMEX`, `LSE`, `TSX`, `ASX` → TradingView.
- Otherwise → Futunn (default).

E Full LLM Prompts

For reproducibility we reproduce the verbatim prompts deployed in production. All prompts are stored in `backend/prompts.yaml` and loaded at server startup; the file must be re-loaded after editing. The full text of the prompts referenced in Section 3.1 follows below.

E.1 Intent Routing Prompt

```
1 SYSTEM (routing.system):
2 You are a precise financial analysis intent router. Always respond with valid
3 JSON only. No markdown fences, no extra text.
4
5 USER (routing.prompt + user_message):
6 You are a financial analysis intent router. Your job is to classify a user
7 query and extract key entities.
8
9 Respond ONLY with valid JSON (no markdown, no explanation). The JSON must
10 have these exact keys:
11 {
12   "mode":          "company_info" | "sector_analysis" | "stock_analysis" | "none",
13   "stock_name":    "the stock/company name mentioned, or empty string",
14   "sector_name":   "the sector name mentioned, or empty string",
15   "reasoning":     "one sentence explaining why you chose this mode"
16 }
17
18 Rules:
19 - "company_info":    user asks about a specific company's profile, financials,
20                     stock price, or description. Look for a stock name or
21                     company name.
22 - "sector_analysis": user asks about sectors, industries, market heatmaps,
23                     sector performance, or which sector to invest in.
24 - "stock_analysis":  user asks whether a stock is worth buying/selling, wants
25                     a prediction, asks about sentiment, wants to "analyze" a
26                     stock, asks "should I invest in X", or asks about a
27                     stock's outlook. Triggers the full ML pipeline.
28 - "none":           general conversation, greetings, or questions that don't
29                     need tools.
30
31 Examples:
32 - "Tell me about Xiaomi" -> {"mode":"company_info", "stock_name":"xiaomi", ...}
33 - "What is the AI sector performance today?" -> {"mode":"sector_analysis", ...}
34 - "Should I buy Xiaomi?" -> {"mode":"stock_analysis", "stock_name":"xiaomi", ...}
35 - "How are you doing?" -> {"mode":"none", ...}
```

E.2 Response-Generation System Prompt

```
1 SYSTEM (system_response):
2 You are FinBot, a professional Hong Kong / global financial analysis assistant.
3
4 Rules:
5 - Answer the user's specific question FIRST and directly. Do NOT dump all
6   available data.
7 - Only mention additional data points if they meaningfully support or
8   contextualise the answer.
9 - Keep responses concise -- 2-4 sentences for a specific question, or a short
10  paragraph for a broader one.
11 - Use bullet points for data only when comparing multiple items (e.g.\ sector
12  rankings).
13 - If you don't have enough data to answer, say so honestly.
14 - Do NOT make up numbers or statistics. Only use data provided in the context.
15 - The dashboard panel already shows full details -- do not repeat data the
16   user can see there.
```

E.3 News Labelling System Prompt

```

1 SYSTEM (labelling.system):
2 You are a precise financial entity and relationship extraction system.
3 Your task is to analyze financial news headlines and map them to our
4 proprietary Financial Knowledge Graph ontology.
5
6 [PASTE YOUR FULL LABELLING PROMPT HERE, INCLUDING THE UNKNOWN ENTITY PROTOCOL]
7
8 USER (labelling.prompt):
9 Headline to analyze: {news_title}

```

E.4 Mode-Specific Response Templates

```

1 USER (response_templates.company_info):
2 You are answering a user question about a company. The dashboard beside this
3 response shows the full company profile, financials, and key statistics.
4
5 User's question: {user_message}
6
7 Retrieved company data:
8 {tool_data}
9
10 Instructions:
11 - Answer the user's question directly and concisely (2-4 sentences).
12 - If the user asked a specific metric (e.g.\ PE ratio, market cap), give that
13   number with a brief interpretation.
14 - If the user asked a broad question, give a brief 2-3 sentence overview.
15 - Reference the dashboard for full details -- do not reproduce everything here.
16
17 USER (response_templates.sector_analysis):
18 You are answering a user question about sector/industry performance. The
19 dashboard beside this response shows the full sector heatmap.
20
21 User's question: {user_message}
22 Retrieved sector data: {tool_data}
23 [ ... instructions identical in spirit to company_info ... ]
24
25 USER (response_templates.stock_analysis):
26 You are answering a user question about a stock's investment outlook. The
27 dashboard beside this response shows the full analysis.
28
29 User's question: {user_message}
30 Retrieved analysis data: {tool_data}
31
32 Instructions:
33 - Extract the signal and probability_up value from the tool_data JSON:
34   * signal: "BUY" or "SELL" (from model_prediction.signal)
35   * probability_up: float in [0, 1] (from model_prediction.probability_up)
36   * price_summary: recent price data (from price_summary)
37 - ALWAYS report the model's signal and probability clearly, e.g.\ "The model
38   predicts a BUY signal with 54.65% probability of upside."
39 - If the user asked about SHORTING:
40   * A BUY signal (upside) means the stock is expected to go UP -- shorting
41     is risky.
42   * A SELL signal means the stock is expected to go DOWN -- shorting aligns
43     with the trend.
44 - Use the model's prediction as the authoritative answer -- do NOT derive
45   your own BUY/SELL from raw news or sentiment scores.
46 - Briefly explain the key factors the model used (recent sentiment trend,
47   notable headlines, price momentum).
48 - Reference the dashboard for full news details and technical indicators.

```

F Stock-Analysis Pipeline Data Flow

The seven-step `stock_analysis` pipeline (`tools/stock_analysis.py run_prediction()`) is implemented as an asynchronous generator that yields a `ThinkingStep` after each phase. The phase-by-phase data flow is:

1. **News scraping** (Futunn, default 20 days, max 2/day) → `news_items = {title, time, source, link, short_description}`.
2. **LLM labelling** (Gemini 2.5 Flash Lite, semaphore = 5) → `relation_id, fixed_sentiment_applicable, related_company, chain_of_thought, confidence_score`.
3. **Sentiment scoring** (FinBERT → TextBlob fallback) → `sentiment_label, raw_sentiment_score, positive_prob, negative_prob, neutral_prob`.
4. **Ontology adjustment**. Competitor mentioned → invert sentiment; supplier/index/match → pass through unchanged. The deployed entity map covers 146 HK-listed tickers with at least one entity link.
5. **Daily aggregation** (20-day lookback) → `sentiment_mean, sentiment_lag_{1,2,3}, news_count, news_lag_{1,2,3}`.
6. **Price feature engineering** (yfinance + technical indicators) → `close, Volume, returns, volatility_10d, volume_change, price_range, RSI, MACD, vwap, hsi_volatility`.
7. **Ensemble prediction** (GBM + LSTM + meta-stacker) → `signal ∈ {BUY, SELL}` and `probability_up ∈ [0, 1]`.

The full result also carries the `ohlcv_data` for the candlestick chart and a `prediction_bar` payload that drives the next-day arrow on the price chart in the `StockAnalysisDashboard`.

G System Architecture Diagrams

This appendix reproduces the four principal architectural diagrams of the Financial Ontology Chatbot system. All diagrams were generated from the production codebase.

G.1 Request Lifecycle: Sequence Diagram

Figure 6 shows the full request lifecycle from user prompt to dashboard render across all four actors: User, Frontend (Next.js), Backend (FastAPI), and LLM (Gemini 2.5 Flash Lite).

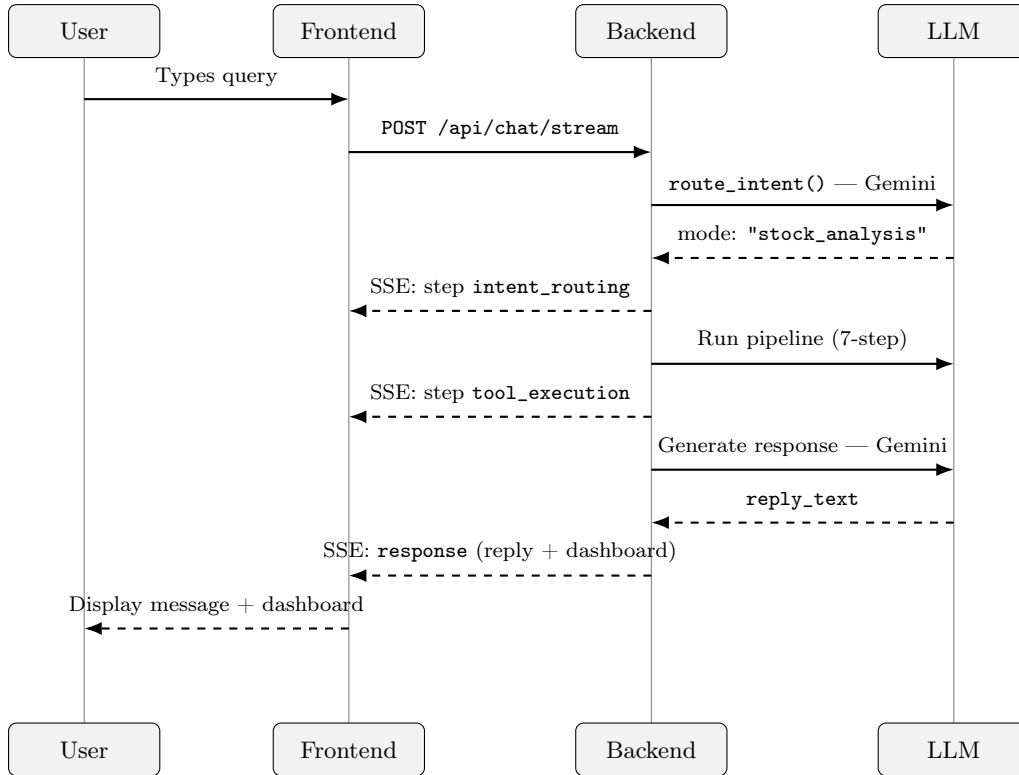


Figure 6: Request lifecycle sequence diagram. Dashed arrows represent response/callback messages; solid arrows represent initiating calls. SSE step events arrive at the Frontend progressively, giving users a live view of each pipeline phase.

G.2 ML Pipeline: Stock-Analysis Flowchart

Figure 7 shows the seven-step `stock_analysis` pipeline from user query to prediction signal.

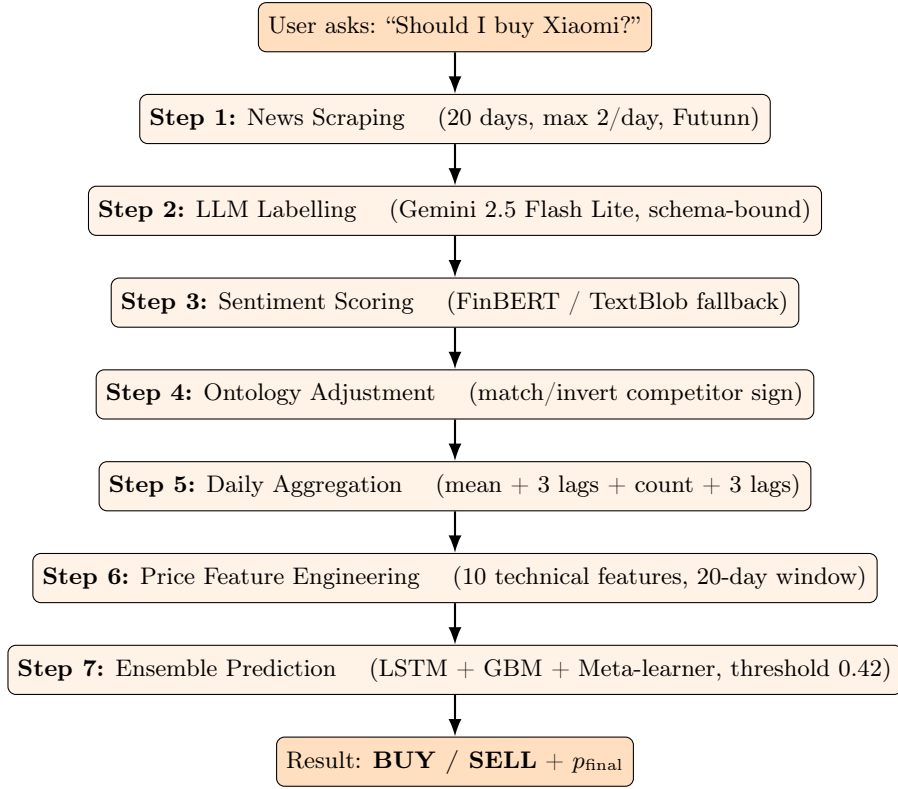


Figure 7: The seven-step `stock_analysis` pipeline. Each step emits a `ThinkingStep` SSE event to the frontend, giving the user a live progress view. Steps 1–2 are network-bound ($\sim 20\text{--}30$ s); Steps 3–7 are compute-bound (~ 3 s total).

G.3 SSE Streaming Flow

Figure 8 details how typed `ThinkingStep` events flow from the async generator in the backend to the React state in the frontend.

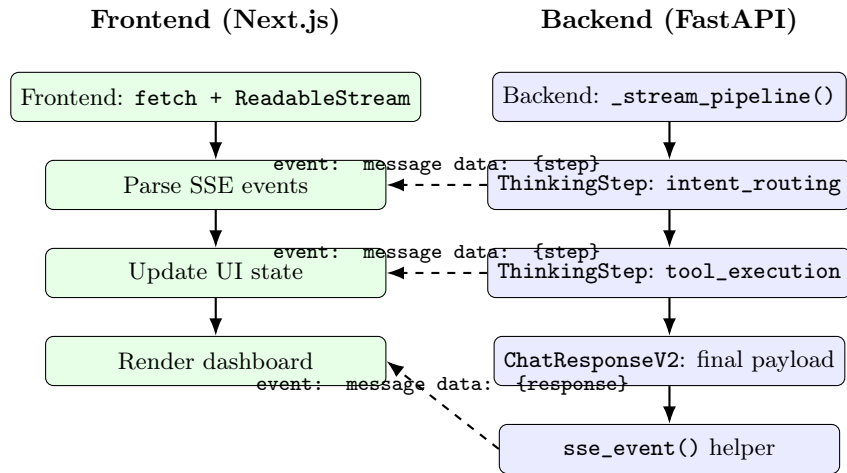


Figure 8: SSE streaming flow. The backend `_stream_pipeline()` async generator yields typed `ThinkingStep` dicts that are formatted as SSE frames by the `sse_event()` helper. The frontend `ReadableStream` reader parses the frames and updates React state incrementally.

H Project Repository

The complete source code for the Financial Ontology Chatbot is publicly available at:

<https://github.com/shl0402/finbot>

The repository contains four top-level directories: **backend/** (FastAPI server, intent router, response generator), **frontend/** (Next.js application, dashboards, streaming client), **tools/** (scrapers, ontology, sentiment pipeline, ensemble model), and **FinBot/** (the legacy Streamlit prototype, retained for reference). Deployment commands, environment-variable reference, and a deeper architecture walkthrough are in the repository's **APPENDIX.md**.

I Web Interface Examples

This appendix provides a comprehensive visual overview of the Next.js frontend, demonstrating the tree-state chat history, the streaming thinking process, and the mode-specific analytical dashboards.

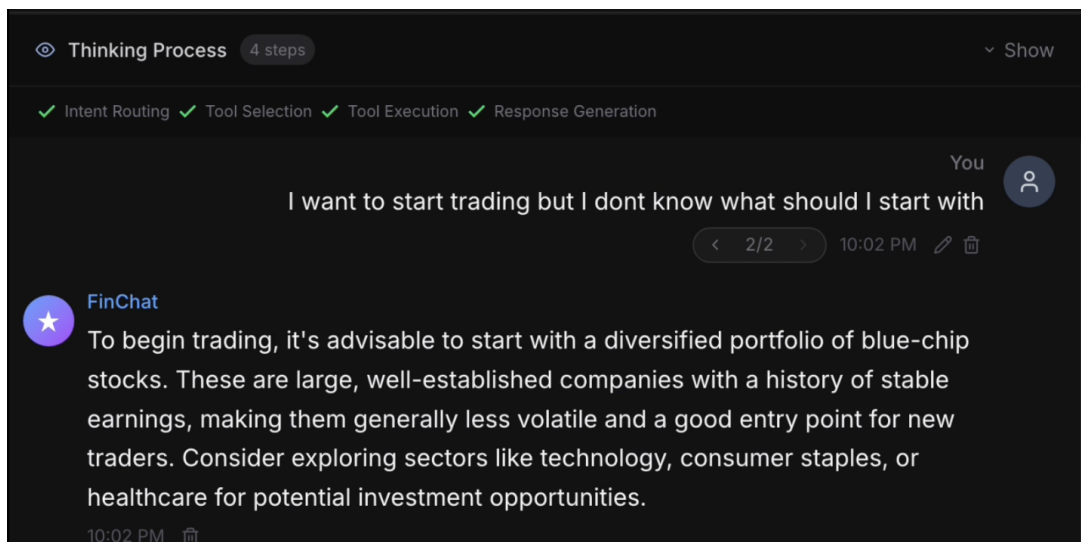


Figure 9: Tree-state chat history interface, illustrating branch navigation for edited prompts.

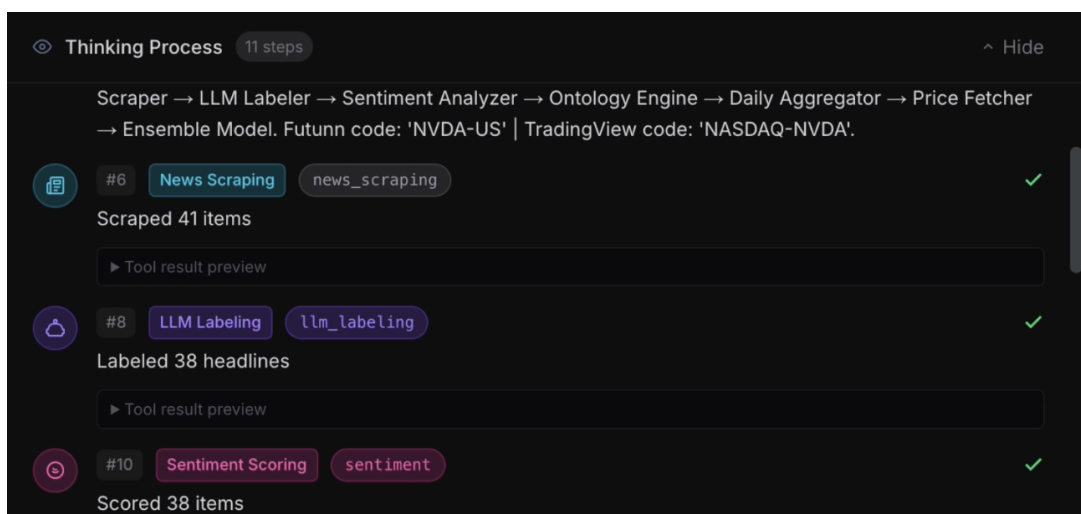


Figure 10: The streaming thinking process component: Execution pipeline initialization.

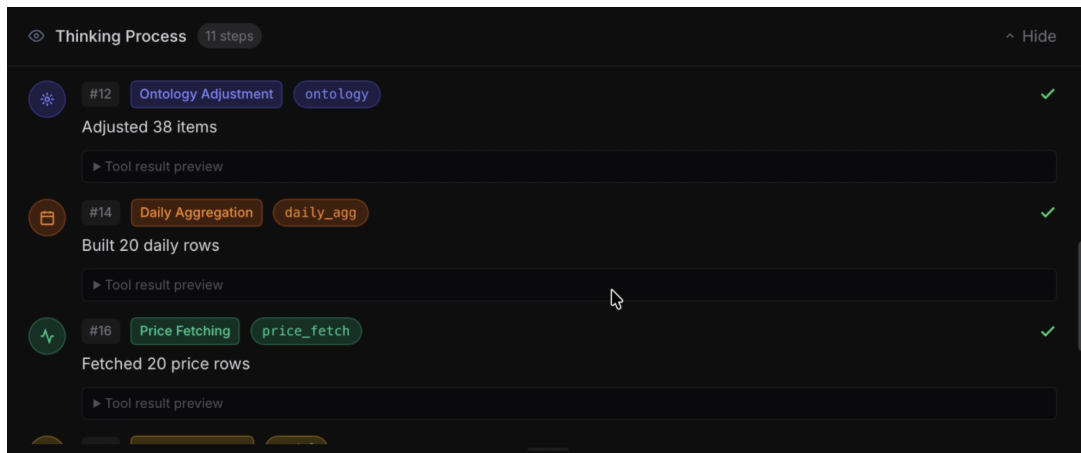


Figure 11: The streaming thinking process component: Tool payloads and intermediate results.

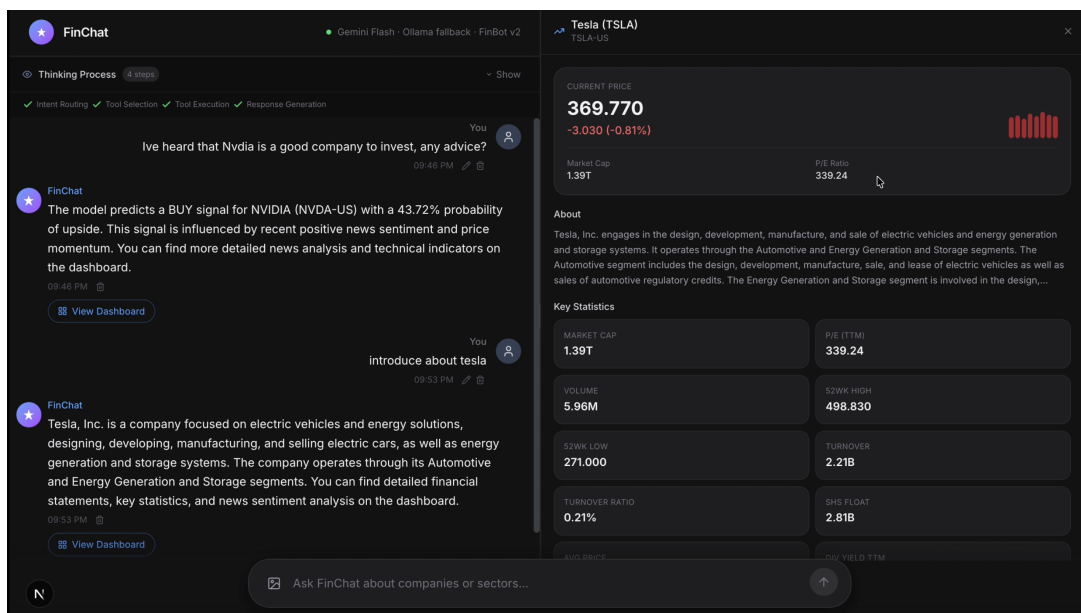


Figure 12: Company Info Dashboard rendering the extracted profile, key statistics, and pricing data.



Figure 13: Sector Dashboard: Sector Overview (hover).



Figure 14: Sector Dashboard: Pie Chart of Top 8 Sector

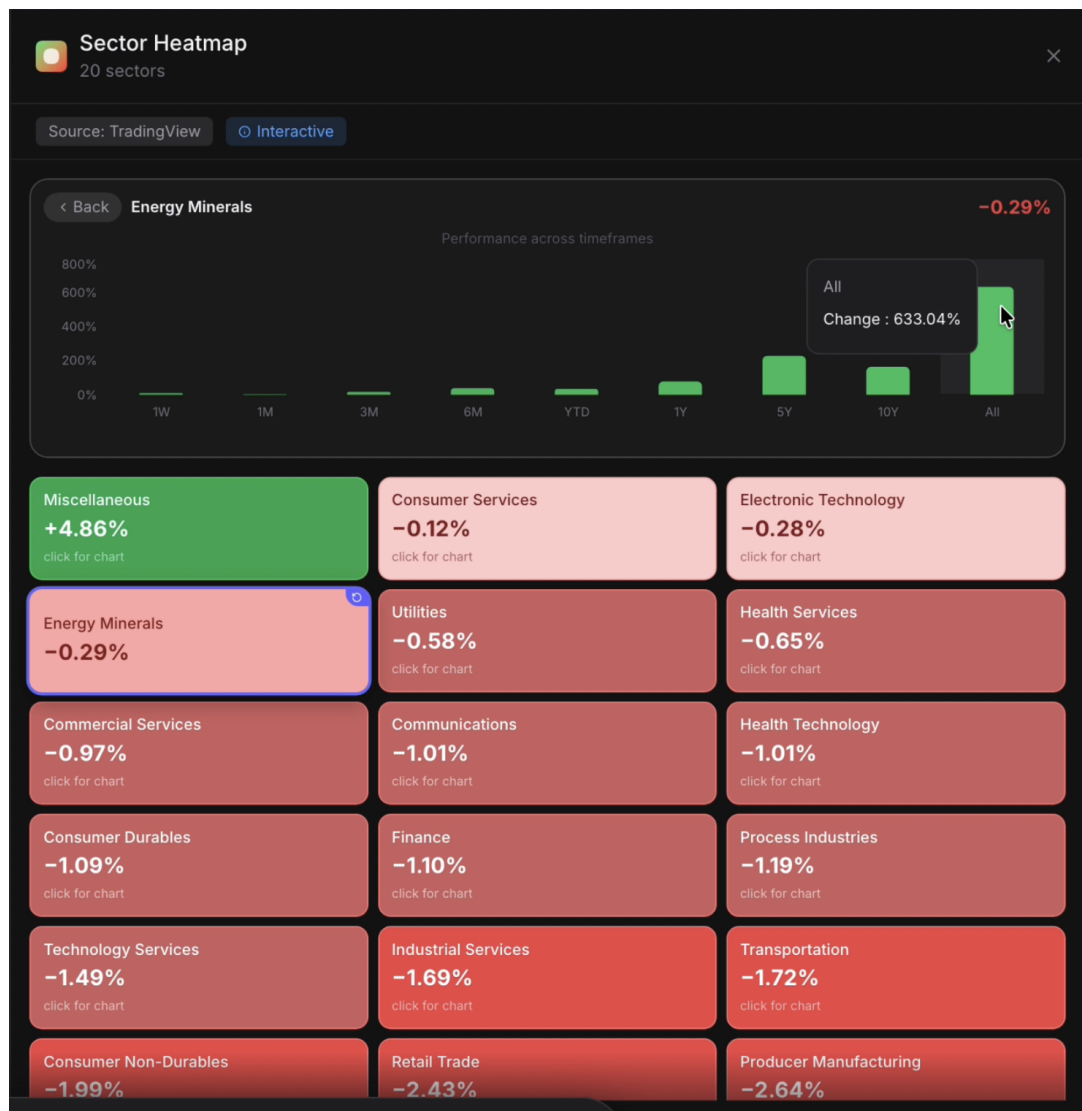


Figure 15: Sector Dashboard: Bar Chart of Price changes

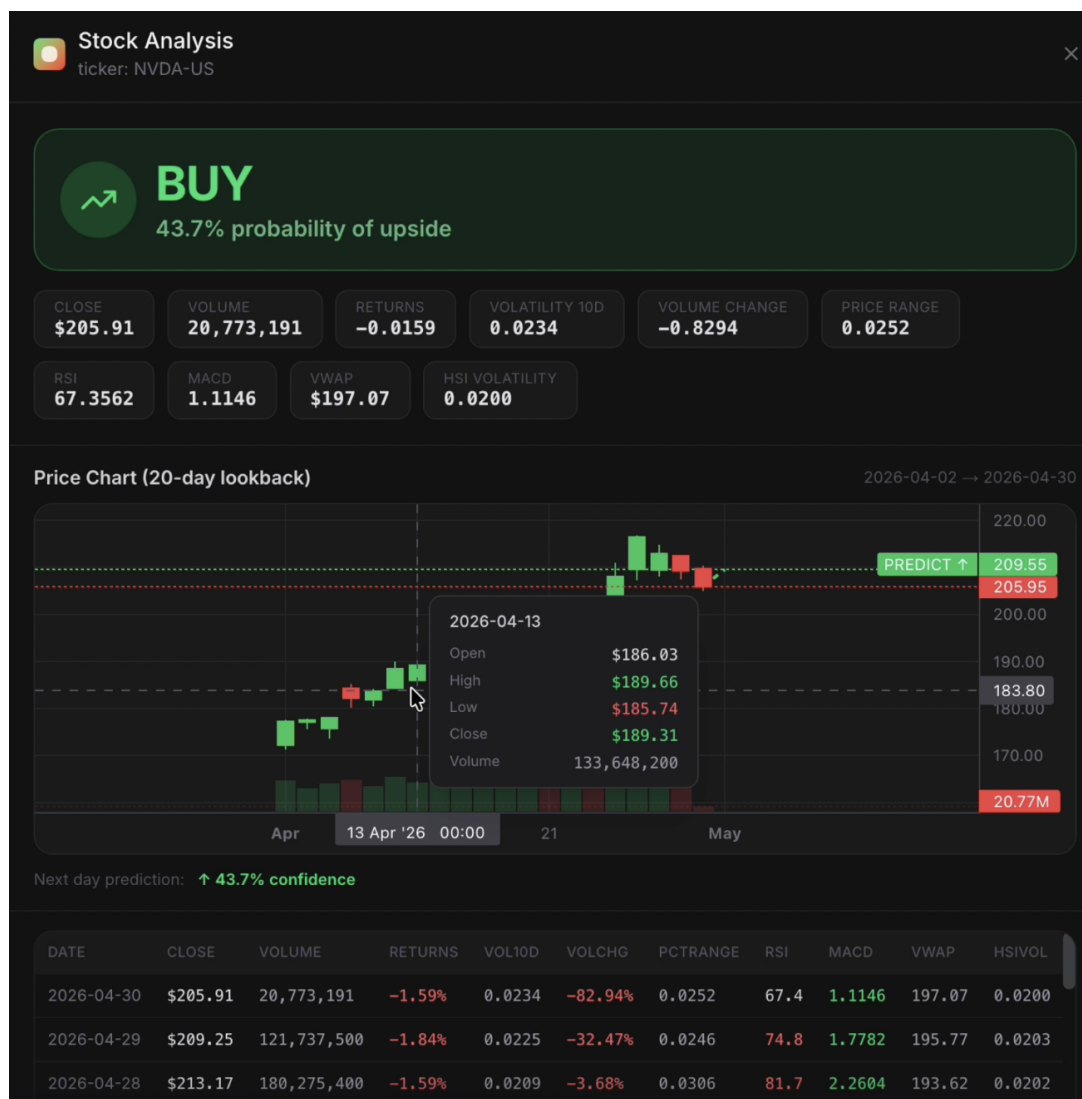


Figure 16: Stock Analysis Dashboard: Technical candlestick chart and BUY/SELL prediction signal.

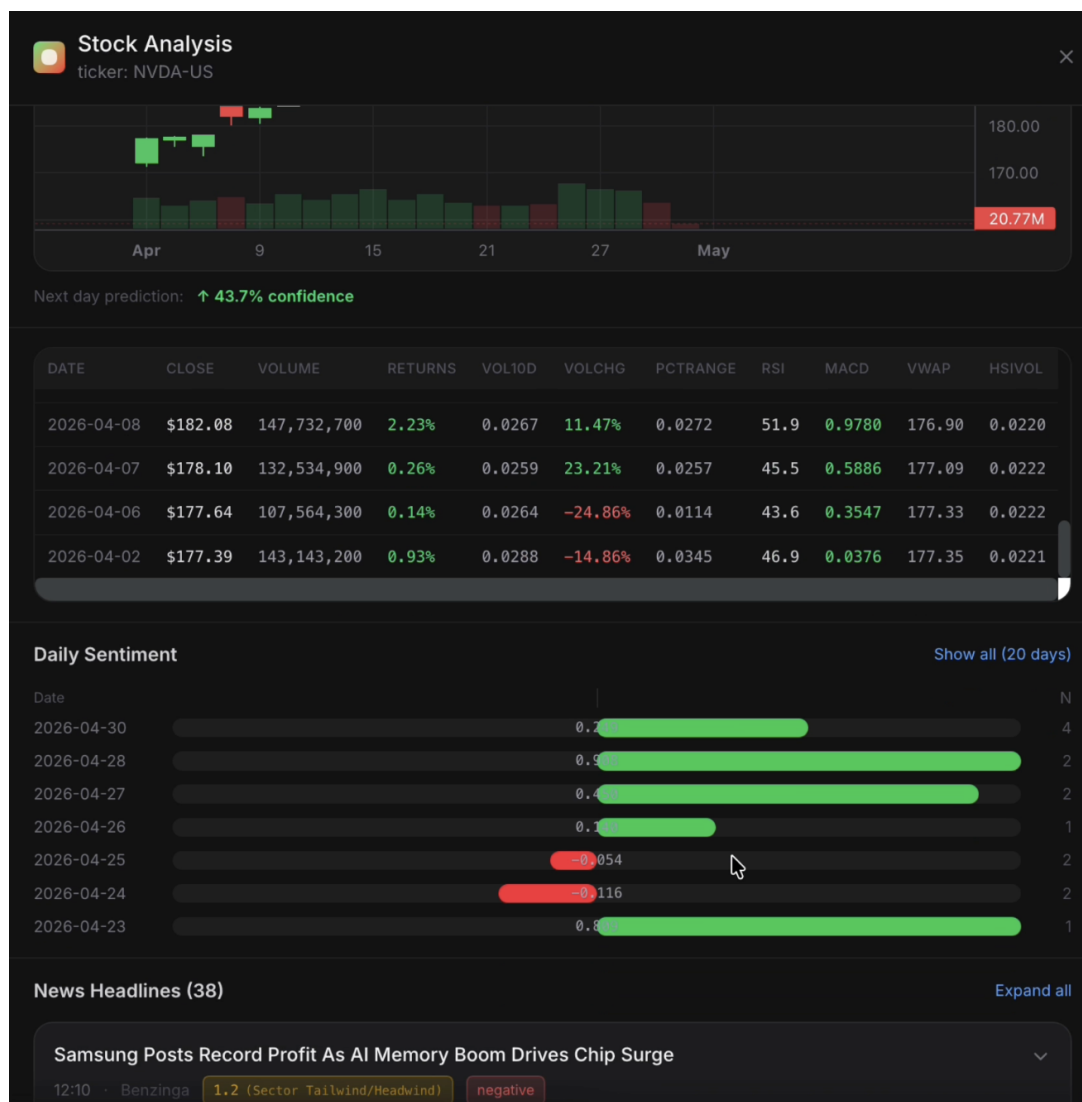


Figure 17: Stock Analysis Dashboard: Sentiment timeline tracking.

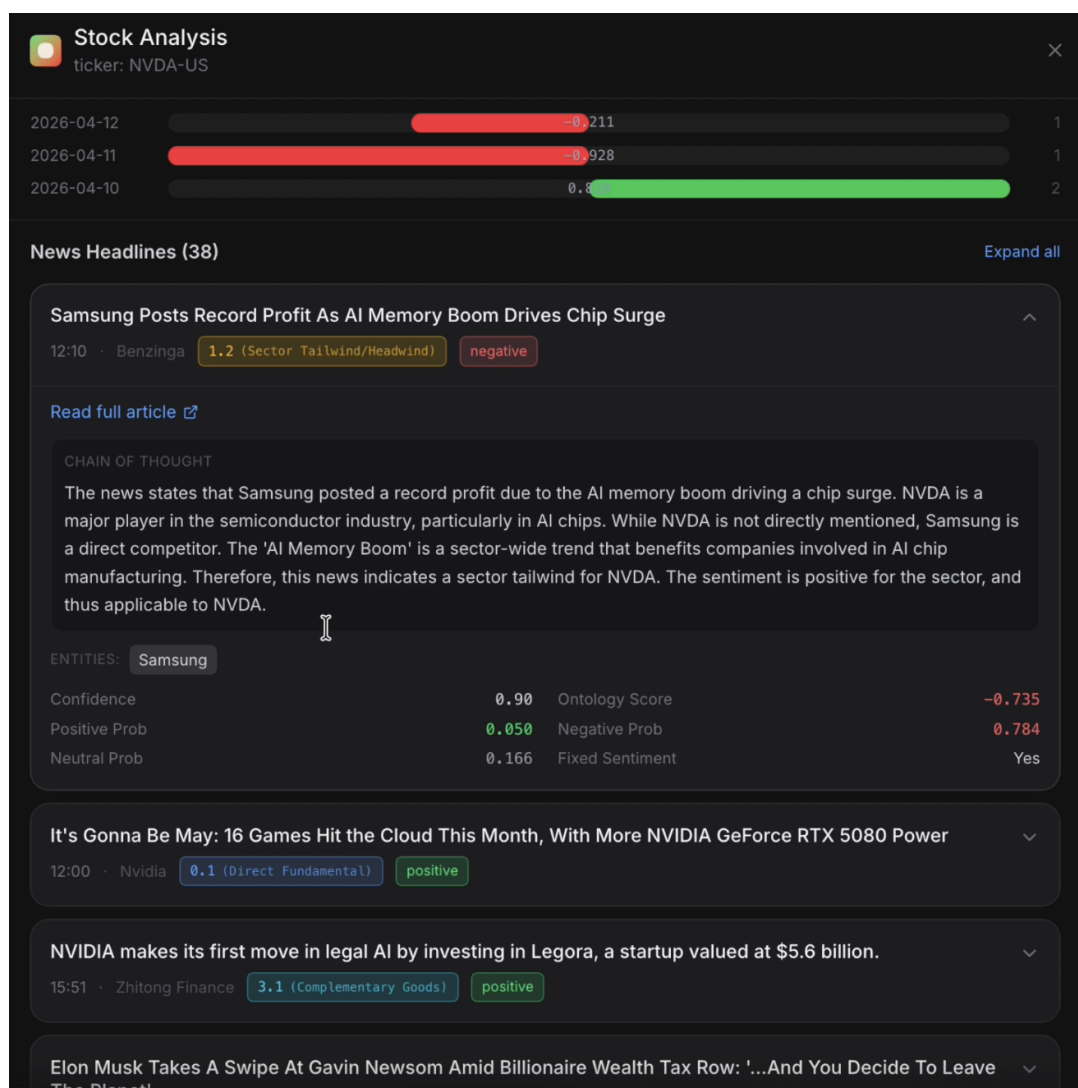


Figure 18: Stock Analysis Dashboard: Relation-driven ontology news labels and chain-of-thought analysis.

J Division of Labour

This project was a three-person collaboration. The table below summarises each member’s primary contributions across the system’s four pillars: (i) the ontology design and ML pipeline, (ii) the scraper fleet and pre-training data acquisition, (iii) the FastAPI backend and Gemini agent integration, and (iv) the Next.js frontend, dashboards, and streaming UX. Tasks were not strictly partitioned—all members contributed to ideation, debugging, and the written report—but ownership, in the sense of “the person primarily responsible for the design choices and code in this area,” is recorded as follows. Estimated effort is given as percentage of total project hours.

Table 8: Division of labour across the three project members. *Primary owner* indicates the team member who led the design and implementation; *Supporting contributors* indicates members who reviewed, integrated, or contributed code/text in that area.

Workstream	Description	Primary owner	Supporting
ML modelling & ontology	Two-branch LSTM design, GBM on flattened aggregates, stacking meta-learner, threshold tuning, ablation and ontology-impact experiments, the 5-domain hierarchy and entity map.	Chris Tang	Harsh, Samuel
Pre-training data	5-iteration scraper design, Futunn headline-only extraction, TradingView Enterprise price export, JSONL storage, smart-resume and CAPTCHA handling, 605,850-headline corpus build.	Harsh	Chris, Samuel
Backend agent & LLM integration	FastAPI server, intent routing (Gemini 2.5 Flash Lite), response-generation prompts, structured-output schema, asyncio bounded concurrency, SSE streaming endpoint, fallback chains.	Samuel	Harsh, Chris
Real-time scraping fleet	Playwright + stealth configuration, anti-bot fingerprint masking, sector heatmap agents, deep stock analytics, live news agent, entity mapping with RapidFuzz.	Harsh	Samuel
Frontend & UX	Next.js 16 application, tree-state chat history, streaming <code>ThinkingProcess</code> component, mode-specific dashboards, candlestick chart with prediction arrow, sentiment timeline.	Samuel	Chris
Dashboard payload contracts	Pydantic schemas (<code>CompanyInfoPayload</code> , <code>SectorPayload</code> , <code>StockAnalysisPayload</code>); backend-frontend payload generators in <code>dashboard_gen.py</code> .	Samuel	Harsh
Experimental evaluation	Multi-horizon evaluation framework, validation-split protocol, threshold tuning experiments, raw-vs-ontology comparison, statistical reporting.	Chris Tang	Harsh, Samuel
Written report	Sections 1–6 drafting, literature review, methodology write-up, experimental results, discussion, limitations, conclusions; figures and tables; $\LaTeX$ typesetting.	All members	—
Estimated total effort share		Chris 33 %	Harsh 33 % Samuel 33 %

The official APAI3799 Work Distribution Form, signed by all project members, is attached below.

THE UNIVERSITY OF HONG KONG

APAI3799 Directed studies in Applied AI Work Distribution Form

I. WORK DISTRIBUTION

Students should define the responsible parts of each group member in detail. If certain chapters or parts of the report are written by more than one member, the proportion of the contribution of each member for that particular chapter / part should be indicated in percentage. If the space provided is insufficient, students may use multiple copies of the same form, all of which should be signed by all members. *This form should be inserted at the end of your written report.*

Name		Responsible Chapters / Parts (%)
1.	Chris Tang	ML modelling & ontology (Primary) Experimental evaluation (Primary) 33%
2.	Harsh	Pre-training data (Primary) Real-time scraping fleet (Primary) 33%
3.	Samuel Hui	Backend agent & LLM integration (Primary) Frontend & UX (Primary) 33%
4.		

II. STUDENTS' SIGNATURE

1. _____

2. _____

3. _____


4. _____

Date: _____ 7/5